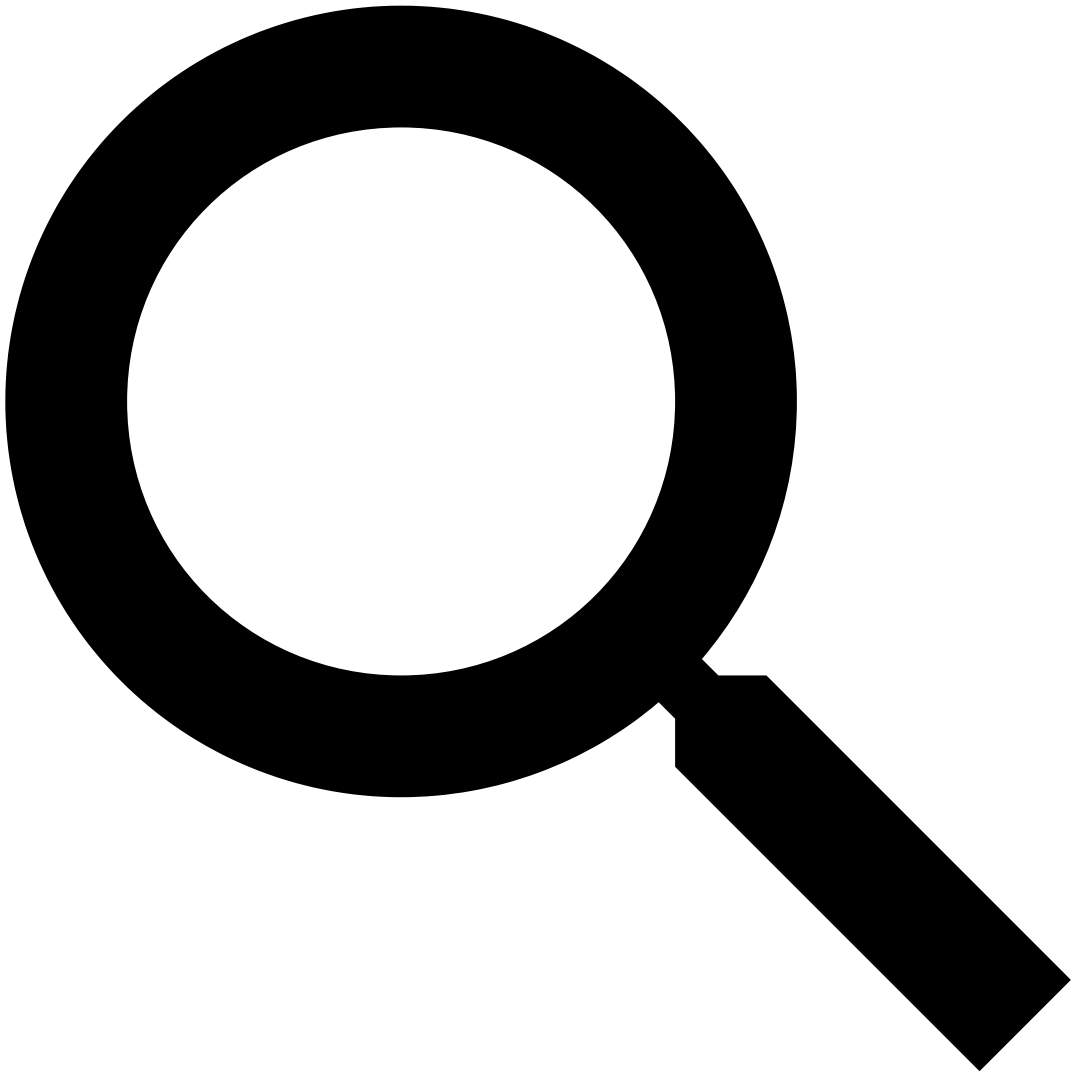
Skip to content

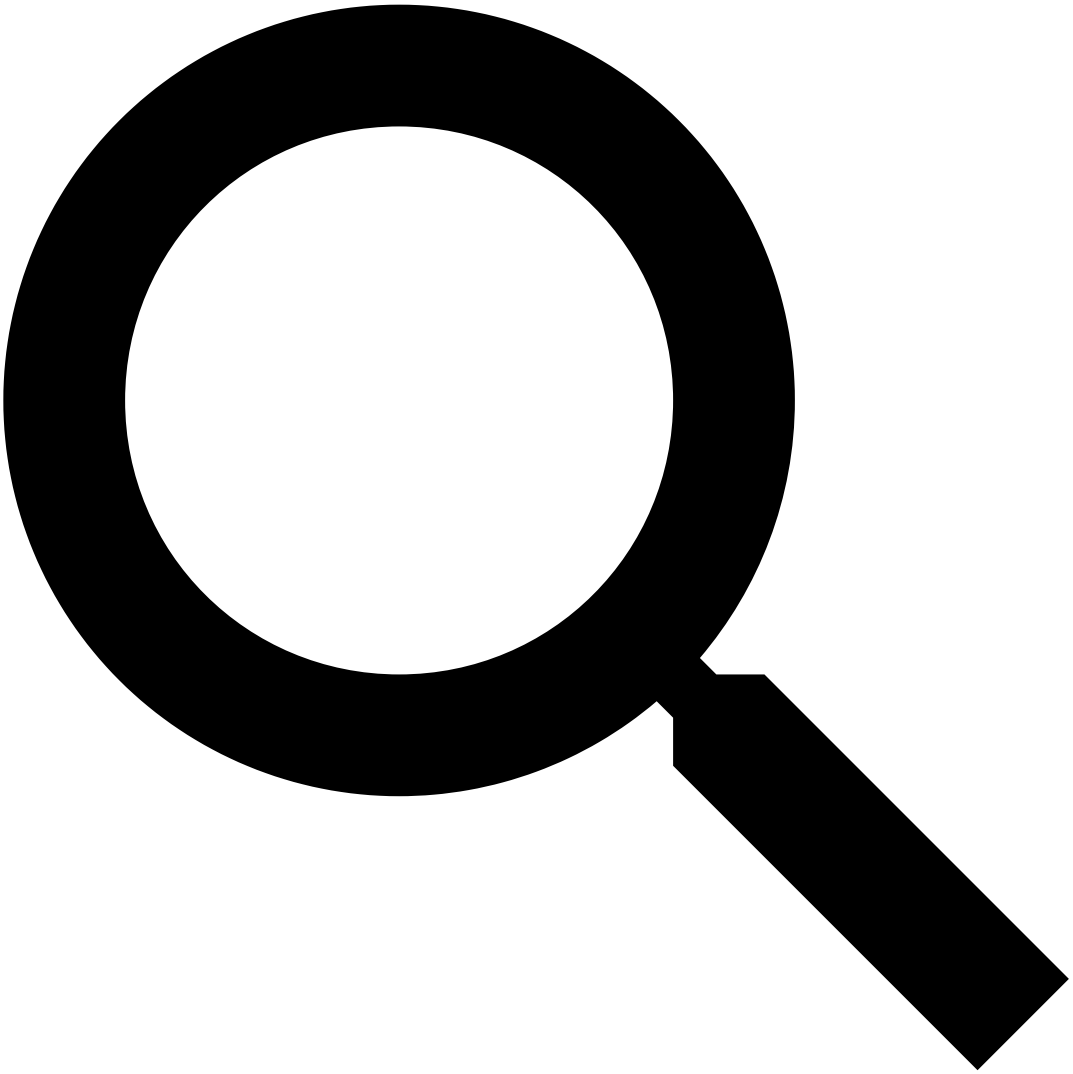
Join the DeepLearning.AI course to explore the **A2A Protocol** Enroll for free

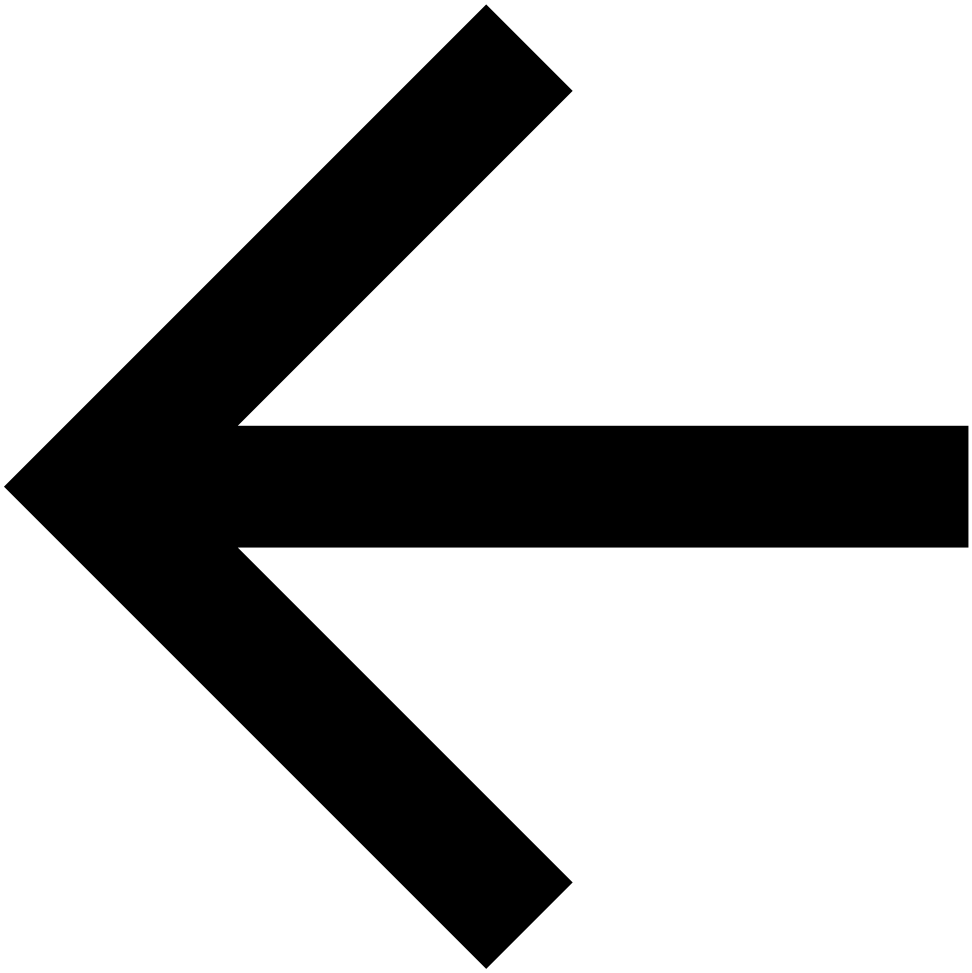
logo

A2A Protocol
Overview

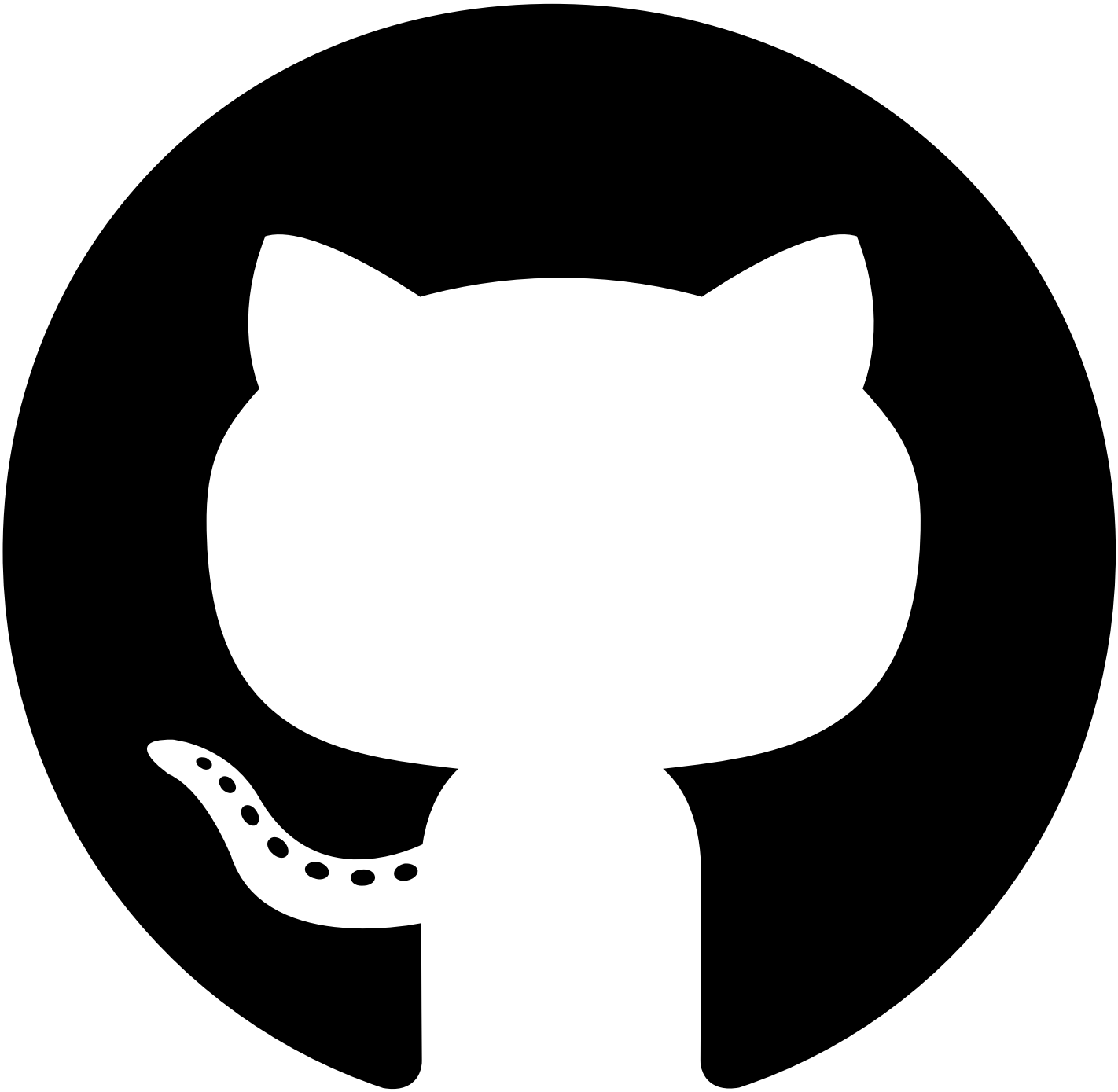


Search





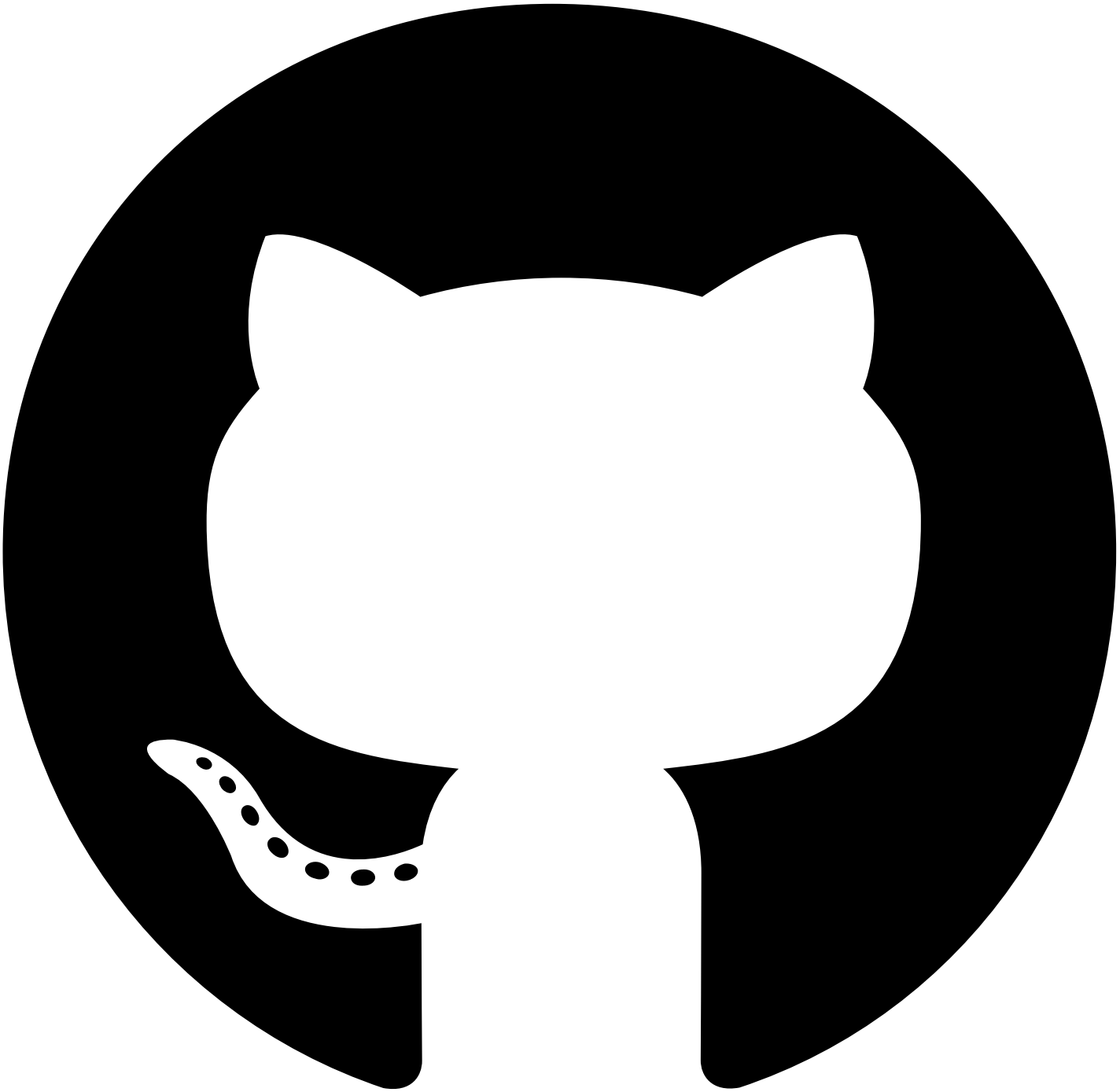
Initializing search



a2aproject/A2A

- Home
- Documentation
- Extensions
- Specification
- Resources
- Community
- Blog

logo A2A Protocol

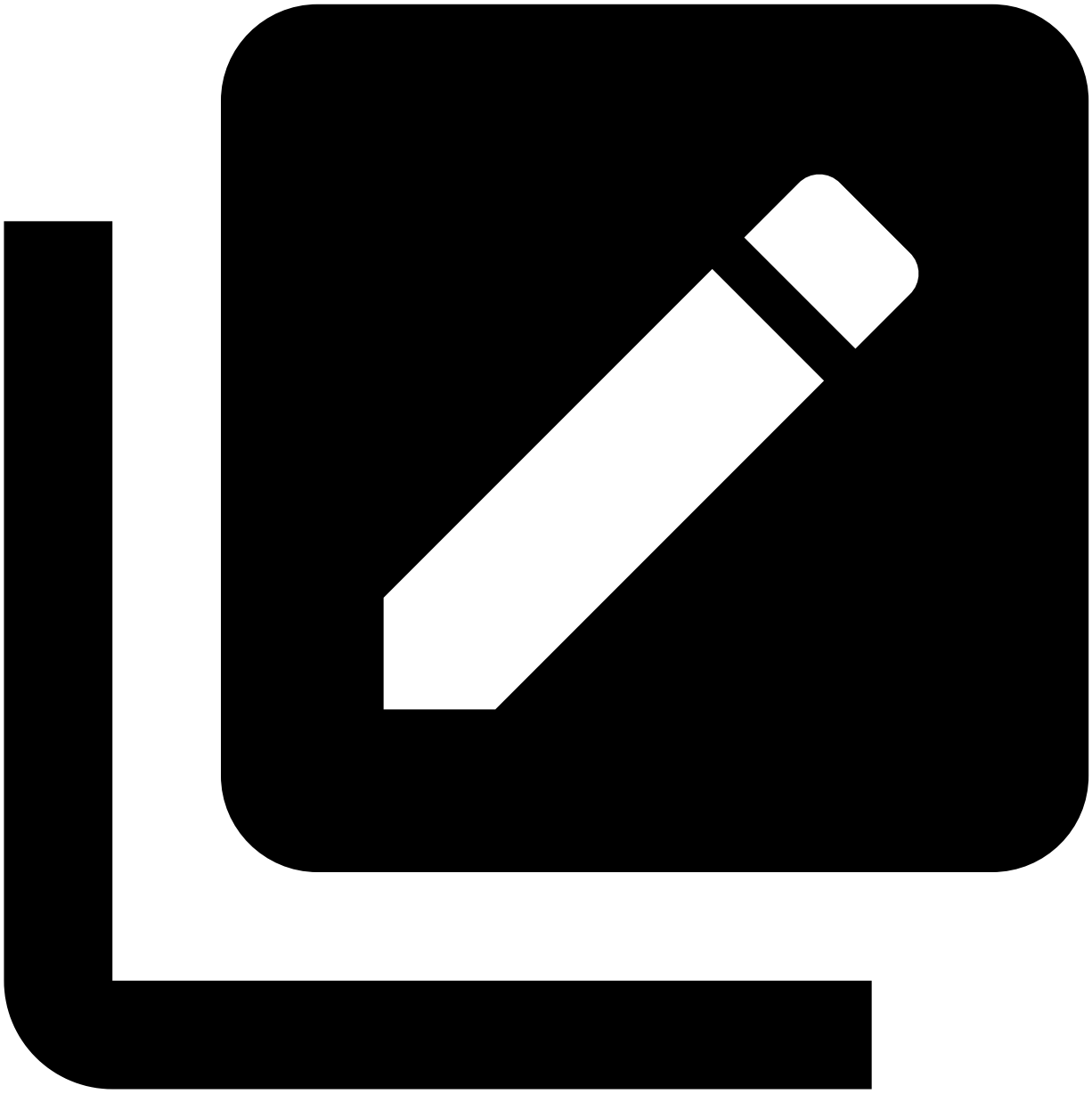


a2aproject/A2A

- ☐ Overview Overview

Table of contents

Table of contents



Agent2Agent (A2A) Protocol Specification¶

▶ **Latest Released Version** `1.0.0`

See Release Notes for changes made between versions.

1. Introduction¶

The Agent2Agent (A2A) Protocol is an open standard designed to facilitate communication and interoperability between independent, potentially opaque AI agent systems. In an ecosystem where agents might be built using different frameworks, languages, or by different vendors, A2A provides a common language and interaction model.

This document provides the detailed technical specification for the A2A protocol. Its primary goal is to enable agents to:

- Discover each other's capabilities.
- Negotiate interaction modalities (text, files, structured data).
- Manage collaborative tasks.

- Securely exchange information to achieve user goals **without needing access to each other's internal state, memory, or tools.**

1.1. Key Goals of A2A¶

- **Interoperability:** Bridge the communication gap between disparate agentic systems.
- **Collaboration:** Enable agents to delegate tasks, exchange context, and work together on complex user requests.
- **Discovery:** Allow agents to dynamically find and understand the capabilities of other agents.
- **Flexibility:** Support various interaction modes including synchronous request/response, streaming for real-time updates, and asynchronous push notifications for long-running tasks.
- **Security:** Facilitate secure communication patterns suitable for enterprise environments, relying on standard web security practices.
- **Asynchronicity:** Natively support long-running tasks and interactions that may involve human-in-the-loop scenarios.

1.2. Guiding Principles¶

- **Simple:** Reuse existing, well-understood standards (HTTP, JSON-RPC 2.0, Server-Sent Events).
- **Enterprise Ready:** Address authentication, authorization, security, privacy, tracing, and monitoring by aligning with established enterprise practices.
- **Async First:** Designed for (potentially very) long-running tasks and human-in-the-loop interactions.
- **Modality Agnostic:** Support exchange of diverse content types including text, audio/video (via file references), structured data/forms, and potentially embedded UI components (e.g., iframes referenced in parts).
- **Opaque Execution:** Agents collaborate based on declared capabilities and exchanged information, without needing to share their internal thoughts, plans, or tool implementations.

For a broader understanding of A2A's purpose and benefits, see What is A2A?.

1.3. Specification Structure¶

This specification is organized into three distinct layers that work together to provide a complete protocol definition:

```
graph TB
    subgraph L1 ["A2A Data Model"]
        direction LR
        A[Task] ~~~ B[Message] ~~~ C[AgentCard] ~~~ D[Part] ~~~ E[Artifact] ~~~ F[Extension]
    end

    subgraph L2 ["A2A Operations"]
        direction LR
        G[Send Message] ~~~ H[Send Streaming Message] ~~~ I[Get Task] ~~~ J[List Tasks] ~~~ K[Cancel Task] ~~~ L[Get Agent Card]
    end

    subgraph L3 ["Protocol Bindings"]
        direction LR
        M[JSON-RPC Methods] ~~~ N[gRPC RPCs] ~~~ O[HTTP/REST Endpoints] ~~~ P[Custom Bindings]
    end

    %% Dependencies between layers
    L1 --> L2
    L2 --> L3


    style A fill:#e1f5fe
    style B fill:#e1f5fe
    style C fill:#e1f5fe
    style D fill:#e1f5fe
    style E fill:#e1f5fe
    style F fill:#e1f5fe

    style G fill:#f3e5f5
    style H fill:#f3e5f5
    style I fill:#f3e5f5
    style J fill:#f3e5f5
    style K fill:#f3e5f5
    style L fill:#f3e5f5

    style M fill:#e8f5e8
    style N fill:#e8f5e8
    style O fill:#e8f5e8

    style L1 fill:#f0f8ff,stroke:#333,stroke-width:2px
    style L2 fill:#faf0ff,stroke:#333,stroke-width:2px
    style L3 fill:#f0fff0,stroke:#333,stroke-width:2px
```

Layer 1: Canonical Data Model defines the core data structures and message formats that all A2A implementations must understand. These are protocol agnostic definitions expressed as Protocol Buffer messages.

Layer 2: Abstract Operations describes the fundamental capabilities and behaviors that A2A agents must support, independent of how they are exposed over specific protocols.

Layer 3: Protocol Bindings provides concrete mappings of the abstract operations and data structures to specific protocol bindings (JSON-RPC, gRPC, HTTP/REST), including method names, endpoint patterns, and protocol-specific behaviors.

This layered approach ensures that:

- Core semantics remain consistent across all protocol bindings
- New protocol bindings can be added without changing the fundamental data model
- Developers can reason about A2A operations independently of binding concerns
- Interoperability is maintained through shared understanding of the canonical data model

1.4 Normative Content¶

In addition to the protocol requirements defined in this document, the file `spec/a2a.proto` is the single authoritative normative definition of all protocol data objects and request/response messages. A generated JSON artifact (`spec/a2a.json`, produced at build time and not committed) MAY be published for convenience to tooling and the website, but it is a non-normative build artifact. SDK language bindings, schemas, and any other derived forms **MUST** be regenerated from the proto (directly or via code generation) rather than edited manually.

Change Control and Deprecation Lifecycle:

- Introduction: When a proto message or field is renamed, the new name is added while existing published names remain available, but marked deprecated, until the next major release.
- Documentation: Migration guidance MUST be provided via an ancillary document when introducing major breaking changes.
- Anchors: Legacy documentation anchors MUST be preserved (as hidden HTML anchors) to avoid breaking inbound links.
- SDK/Schema Aliases: SDKs and JSON Schemas SHOULD provide deprecated alias types/definitions to maintain backward compatibility.
- Removal: A deprecated name SHOULD NOT be removed earlier than the next major version after introduction of its replacement.

Automated Generation:

The documentation build generates `specification/json/a2a.json` on-the-fly (the file is not tracked in source control). Future improvements may publish an OpenAPI v3 + JSON Schema bundle for enhanced tooling.

Rationale:

Centering the proto file as the normative source ensures protocol neutrality, reduces specification drift, and provides a deterministic evolution path for the ecosystem.

2. Terminology¶

2.1. Requirements Language¶

The keywords "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119.

2.2. Core Concepts¶

A2A revolves around several key concepts. For detailed explanations, please refer to the Key Concepts guide.

- **A2A Client:** An application or agent that initiates requests to an A2A Server on behalf of a user or another system.
- **A2A Server (Remote Agent):** An agent or agentic system that exposes an A2A-compliant endpoint, processing tasks and providing responses.
- **Agent Card:** A JSON metadata document published by an A2A Server, describing its identity, capabilities, skills, service endpoint, and authentication requirements.
- **Message:** A communication turn between a client and a remote agent, having a `role` ("user" or "agent") and containing one or more `Parts`.
- **Task:** The fundamental unit of work managed by A2A, identified by a unique ID. Tasks are stateful and progress through a defined lifecycle.
- **Part:** The smallest unit of content within a Message or Artifact. Parts can contain text, file references, or structured data.
- **Artifact:** An output (e.g., a document, image, structured data) generated by the agent as a result of a task, composed of `Parts`.
- **Streaming:** Real-time, incremental updates for tasks (status changes, artifact chunks) delivered via protocol-specific streaming mechanisms.
- **Push Notifications:** Asynchronous task updates delivered via server-initiated HTTP POST requests to a client-provided webhook URL, for long-running or disconnected scenarios.
- **Context:** An optional identifier to logically group related tasks and messages.
- **Extension:** A mechanism for agents to provide additional functionality or data beyond the core A2A specification.

3. A2A Protocol Operations¶

This section describes the core operations of the A2A protocol in a binding-independent manner. These operations define the fundamental capabilities that all A2A implementations must support, regardless of the underlying binding mechanism.

3.1. Core Operations¶

The following operations define the fundamental capabilities that all A2A implementations must support, independent of the specific protocol binding used. For a quick reference mapping of these operations to protocol-specific method names and endpoints, see Section 5.3 (Method Mapping Reference). For detailed protocol-specific implementation details, see:

- Section 9: JSON-RPC Protocol Binding
- Section 10: gRPC Protocol Binding
- Section 11: HTTP+JSON/REST Protocol Binding

3.1.1. Send Message¶

The primary operation for initiating agent interactions. Clients send a message to an agent and receive either a task that tracks the processing or a direct response message.

Inputs:

- `SendMessageRequest`: Request object containing the message, configuration, and metadata

Outputs:

- `Task`: A task object representing the processing of the message, OR
- `Message`: A direct response message (for simple interactions that don't require task tracking)

Errors:

- `ContentTypeNotSupportedError`: A Media Type provided in the request's message parts is not supported by the agent.
- `UnsupportedOperationError`: Messages sent to Tasks that are in a terminal state (`TASK_STATE_COMPLETED`, `TASK_STATE_FAILED`, `TASK_STATE_CANCELED`, `TASK_STATE_REJECTED`) cannot accept further messages.

- `TaskNotFoundError`: The task ID does not exist or is not accessible.

Behavior:

The agent MAY create a new `Task` to process the provided message asynchronously or MAY return a direct `Message` response for simple interactions. The operation MUST return immediately with either task information or response message. Task processing MAY continue asynchronously after the response when a `Task` is returned.

3.1.2. Send Streaming Message¶

Similar to Send Message but with real-time streaming of updates during processing.

Inputs:

- `SendMessageRequest`: Request object containing the message, configuration, and metadata

Outputs:

- `Stream Response` object containing:
  - Initial response: `Task` object OR `Message` object
  - Subsequent events following a `Task` MAY include stream of `TaskStatusUpdateEvent` and `TaskArtifactUpdateEvent` objects
- Final completion indicator

Errors:

- `UnsupportedOperationError`: Streaming is not supported by the agent (see Capability Validation).
- `UnsupportedOperationError`: Messages sent to Tasks that are in a terminal state (`TASK_STATE_COMPLETED`, `TASK_STATE_FAILED`, `TASK_STATE_CANCELED`, `TASK_STATE_REJECTED`) cannot accept further messages.
- `ContentTypeNotSupportedError`: A Media Type provided in the request's message parts is not supported by the agent.
- `TaskNotFoundError`: The task ID does not exist or is not accessible.

Behavior:

The operation MUST establish a streaming connection for real-time updates. The stream MUST follow one of these patterns:

1. **Message-only stream:** If the agent returns a `Message`, the stream MUST contain exactly one `Message` object and then close immediately. No task tracking or updates are provided.
2. **Task lifecycle stream:** If the agent returns a `Task`, the stream MUST begin with the Task object, followed by zero or more `TaskStatusUpdateEvent` or `TaskArtifactUpdateEvent` objects. The stream MUST close when the task reaches a terminal state (`TASK_STATE_COMPLETED`, `TASK_STATE_FAILED`, `TASK_STATE_CANCELED`, `TASK_STATE_REJECTED`).

The agent MAY return a `Task` for complex processing with status/artifact updates or MAY return a `Message` for direct streaming responses without task overhead. The implementation MUST provide immediate feedback on progress and intermediate results.

3.1.3. Get Task¶

Retrieves the current state (including status, artifacts, and optionally history) of a previously initiated task. This is typically used for polling the status of a task initiated with Send Message, or for fetching the final state of a task after being notified via a push notification or after a stream has ended.

Inputs:

Represents a request for the `GetTask` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`id`	`string`	Yes	The resource ID of the task to retrieve.
`historyLength`	`integer`	No	The maximum number of most recent messages from the task's history to retrieve. An unset value means the client does not impose any limit. A value of zero is a request to not include any messages. The server MUST NOT return more messages than the provided value, but MAY apply a lower limit.

See History Length Semantics for details about `historyLength`.

Outputs:

- `Task`: Current state and artifacts of the requested task

Errors:

- `TaskNotFoundError`: The task ID does not exist or is not accessible.

3.1.4. List Tasks¶

Retrieves a list of tasks with optional filtering and pagination capabilities. This method allows clients to discover and manage multiple tasks across different contexts or with specific status criteria.

Inputs:

Parameters for listing tasks with optional filtering criteria.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`contextId`	`string`	No	Filter tasks by context ID to get tasks from a specific conversation or session.
`status`	`TaskState`	No	Filter tasks by their current status state.
`pageSize`	`integer`	No	The maximum number of tasks to return. The service may return fewer than this value. If unspecified, at most 50 tasks will be returned. The minimum value is 1. The maximum value is 100.
`pageToken`	`string`	No	A page token, received from a previous `ListTasks` call. `ListTasksResponse.next_page_token`. Provide this to retrieve the subsequent page.
`historyLength`	`integer`	No	The maximum number of messages to include in each task's history.
`statusTimestampAfter`	`timestamp`	No	Filter tasks which have a status updated after the provided timestamp in ISO 8601 format (e.g., "2023-10-27T10:00:00Z"). Only tasks with a status timestamp time greater than or equal to this value will be returned.
`includeArtifacts`	`boolean`	No	Whether to include artifacts in the returned tasks. Defaults to false to reduce payload size.

When `includeArtifacts` is false (the default), the artifacts field MUST be omitted entirely from each Task object in the response. The field should not be present as an empty array or null value. When `includeArtifacts` is true, the artifacts field should be included with its actual content (which may be an empty array if the task has no artifacts).

Outputs:

Result object for `ListTasks` method containing an array of tasks and pagination information.

Field	Type	Required	Description
`tasks`	array of `Task`	Yes	Array of tasks matching the specified criteria.
`nextPageToken`	`string`	Yes	A token to retrieve the next page of results, or empty if there are no more results in the list.
`pageSize`	`integer`	Yes	The page size used for this response.
`totalSize`	`integer`	Yes	Total number of tasks available (before pagination).

Note on `nextPageToken`: The `nextPageToken` field MUST always be present in the response. When there are no more results to retrieve (i.e., this is the final page), the field MUST be set to an empty string (""). Clients should check for an empty string to determine if more pages are available.

Errors:

None specific to this operation beyond standard protocol errors.

Behavior:

The operation MUST return only tasks visible to the authenticated client and MUST use cursor-based pagination for performance and consistency. Tasks MUST be sorted by last update time in descending order. Implementations MUST implement appropriate authorization scoping to ensure clients can only access authorized tasks. See Section 13.1 Data Access and Authorization Scoping for detailed security requirements.

Pagination Strategy:

This method uses cursor-based pagination (via `pageToken`/`nextPageToken`) rather than offset-based pagination for better performance and consistency, especially with large datasets. Cursor-based pagination avoids the "deep pagination problem" where skipping large numbers of records becomes inefficient for databases. This approach is consistent with the gRPC specification, which also uses cursor-based pagination (page_token/next_page_token).

Ordering:

Implementations MUST return tasks sorted by their status timestamp time in descending order (most recently updated tasks first). This ensures consistent pagination and allows clients to efficiently monitor recent task activity.

3.1.5. Cancel Task¶

Requests the cancellation of an ongoing task. The server will attempt to cancel the task, but success is not guaranteed (e.g., the task might have already completed or failed, or cancellation might not be supported at its current stage).

Inputs:

Represents a request for the `CancelTask` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`id`	`string`	Yes	The resource ID of the task to cancel.
`metadata`	`object`	No	A flexible key-value map for passing additional context or parameters.

Outputs:

- Updated `Task` with cancellation status

Errors:

- `TaskNotCancelableError`: The task is not in a cancelable state (e.g., already completed, failed, or canceled).
- `TaskNotFoundError`: The task ID does not exist or is not accessible.

Behavior:

The operation attempts to cancel the specified task and returns its updated state.

3.1.6. Subscribe to Task¶

Establishes a streaming connection to receive updates for an existing task.

Inputs:

Represents a request for the `SubscribeToTask` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`id`	`string`	Yes	The resource ID of the task to subscribe to.

Outputs:

- `Stream Response` object containing:
  - Initial response: `Task` object with current state
  - Stream of `TaskStatusUpdateEvent` and `TaskArtifactUpdateEvent` objects

Errors:

- `UnsupportedOperationError`: Streaming is not supported by the agent (see Capability Validation).
- `TaskNotFoundError`: The task ID does not exist or is not accessible.
- `UnsupportedOperationError`: The operation is attempted on a task that is in a terminal state (`TASK_STATE_COMPLETED`, `TASK_STATE_FAILED`, `TASK_STATE_CANCELED`, `TASK_STATE_REJECTED`).

Behavior:

The operation enables real-time monitoring of task progress and can be used with any task that is not in a terminal state. The stream MUST terminate when the task reaches a terminal state (`TASK_STATE_COMPLETED`, `TASK_STATE_FAILED`, `TASK_STATE_CANCELED`, `TASK_STATE_REJECTED`).

The operation MUST return a `Task` object as the first event in the stream, representing the current state of the task at the time of subscription. This prevents a potential loss of information between a call to `GetTask` and calling `SubscribeToTask`.

3.1.7. Create Push Notification Config¶

Creates a push notification configuration for a task to receive asynchronous updates via webhook.

Inputs:

A container associating a push notification configuration with a specific task.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`id`	`string`	No	The push notification configuration details. A unique identifier (e.g. UUID) for this push notification configuration.
`taskId`	`string`	No	The ID of the task this configuration is associated with.
`url`	`string`	Yes	The URL where the notification should be sent.
`token`	`string`	No	A token unique for this task or session.
`authentication`	`AuthenticationInfo`	No	Authentication information required to send the notification.

Outputs:

- `PushNotificationConfig`: Created configuration with assigned ID

Errors:

- `PushNotificationNotSupportedError`: Push notifications are not supported by the agent (see Capability Validation).
- `TaskNotFoundError`: The task ID does not exist or is not accessible.

Behavior:

The operation MUST establish a webhook endpoint for task update notifications. When task updates occur, the agent will send HTTP POST requests to the configured webhook URL with `StreamResponse` payloads (see Push Notification Payload for details). This operation is only available if the agent supports push notifications capability. The configuration MUST persist until task completion or explicit deletion.

3.1.8. Get Push Notification Config¶

Retrieves an existing push notification configuration for a task.

Inputs:

Represents a request for the `GetTaskPushNotificationConfig` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`taskId`	`string`	Yes	The parent task resource ID.
`id`	`string`	Yes	The resource ID of the configuration to retrieve.

Outputs:

- `PushNotificationConfig`: The requested configuration

Errors:

- `PushNotificationNotSupportedError`: Push notifications are not supported by the agent (see Capability Validation).
- `TaskNotFoundError`: The push notification configuration does not exist.

Behavior:

The operation MUST return configuration details including webhook URL and notification settings. The operation MUST fail if the configuration does not exist or the client lacks access.

3.1.9. List Push Notification Configs¶

Retrieves all push notification configurations for a task.

Inputs:

Represents a request for the `ListTaskPushNotificationConfigs` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`taskId`	`string`	Yes	The parent task resource ID.
`pageSize`	`integer`	No	The maximum number of configurations to return.
`pageToken`	`string`	No	A page token received from a previous `ListTaskPushNotificationConfigsRequest` call.

Outputs:

Represents a successful response for the `ListTaskPushNotificationConfigs` method.

Field	Type	Required	Description
`configs`	array of `TaskPushNotificationConfig`	No	The list of push notification configurations.
`nextPageToken`	`string`	No	A token to retrieve the next page of results, or empty if there are no more results in the list.

Errors:

- `PushNotificationNotSupportedError`: Push notifications are not supported by the agent (see Capability Validation).
- `TaskNotFoundError`: The task ID does not exist or is not accessible.

Behavior:

The operation MUST return all active push notification configurations for the specified task and MAY support pagination for tasks with many configurations.

3.1.10. Delete Push Notification Config¶

Removes a push notification configuration for a task.

Inputs:

Represents a request for the `DeleteTaskPushNotificationConfig` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`taskId`	`string`	Yes	The parent task resource ID.
`id`	`string`	Yes	The resource ID of the configuration to delete.

Outputs:

- Confirmation of deletion (implementation-specific)

Errors:

- `PushNotificationNotSupportedError`: Push notifications are not supported by the agent (see Capability Validation).
- `TaskNotFoundError`: The task ID does not exist.

Behavior:

The operation MUST permanently remove the specified push notification configuration. No further notifications will be sent to the configured webhook after deletion. This operation MUST be idempotent - multiple deletions of the same config have the same effect.

3.1.11. Get Extended Agent Card¶

Retrieves a potentially more detailed version of the Agent Card after the client has authenticated. This endpoint is available only if `AgentCard.capabilities.extendedAgentCard` is `true`.

Inputs:

Represents a request for the `GetExtendedAgentCard` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.

Outputs:

- `AgentCard`: A complete Agent Card object, which may contain additional details or skills not present in the public card

Errors:

- `UnsupportedOperationError`: The agent does not support authenticated extended cards (see Capability Validation).
- `ExtendedAgentCardNotConfiguredError`: The agent declares support but does not have an extended agent card configured.

Behavior:

- **Authentication**: The client MUST authenticate the request using one of the schemes declared in the public `AgentCard.securitySchemes` and `AgentCard.security` fields.
- **Extended Information**: The operation MAY return different details based on client authentication level, including additional skills, capabilities, or configuration not available in the public Agent Card.
- **Card Replacement**: Clients retrieving this extended card SHOULD replace their cached public Agent Card with the content received from this endpoint for the duration of their authenticated session or until the card's version changes.
- **Availability**: This operation is only available if the public Agent Card declares `capabilities.extendedAgentCard: true`.

For detailed security guidance on extended agent cards, see Section 13.3 Extended Agent Card Access Control.

3.2. Operation Parameter Objects¶

This section defines common parameter objects used across multiple operations.

3.2.1. SendMessageRequest¶

Represents a request for the `SendMessage` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`message`	`Message`	Yes	The message to send to the agent.
`configuration`	`SendMessageConfiguration`	No	Configuration for the send request.
`metadata`	`object`	No	A flexible key-value map for passing additional context or parameters.

3.2.2. SendMessageConfiguration¶

Configuration of a send message request.

Field	Type	Required	Description
`acceptedOutputModes`	array of `string`	No	A list of media types the client is prepared to accept for response parts. Agents SHOULD use this to tailor their output.
`taskPushNotificationConfig`	`TaskPushNotificationConfig`	No	Configuration for the agent to send push notifications for task updates. Task id should be empty when sending this configuration in a `SendMessage` request.
`historyLength`	`integer`	No	The maximum number of most recent messages from the task's history to retrieve in the response. An unset value means the client does not impose any limit. A value of zero is a request to not include any messages. The server MUST NOT return more messages than the provided value, but MAY apply a lower limit.
`returnImmediately`	`boolean`	No	If `true`, the operation returns immediately after creating the task, even if processing is still in progress. If `false` (default), the operation MUST wait until the task reaches a terminal (`COMPLETED`, `FAILED`, `CANCELED`, `REJECTED`) or interrupted (`INPUT_REQUIRED`, `AUTH_REQUIRED`) state before returning.

Execution Mode:

The `return_immediately` field in `SendMessageConfiguration` controls whether the operation returns immediately or waits for task completion. Operations are blocking by default:

- **Blocking (`return_immediately: false` or unset)**: The operation MUST wait until the task reaches a terminal state (`TASK_STATE_COMPLETED`, `TASK_STATE_FAILED`, `TASK_STATE_CANCELED`, `TASK_STATE_REJECTED`) or an interrupted state (`TASK_STATE_INPUT_REQUIRED`, `TASK_STATE_AUTH_REQUIRED`) before returning. The response MUST include the latest task state with all artifacts and status information. This is the default behavior.
- **Non-Blocking (`return_immediately: true`)**: The operation MUST return immediately after creating the task, even if processing is still in progress. The returned task will have an in-progress state (e.g., `TASK_STATE_WORKING`, `TASK_STATE_INPUT_REQUIRED`). It is the caller's responsibility to poll for updates using Get Task, subscribe via Subscribe to Task, or receive updates via push notifications.

The `return_immediately` field has no effect:

- when the operation returns a direct `Message` response instead of a task.
- for streaming operations, which always return updates in real-time.
- on configured push notification configurations, which operates independently of execution mode.

3.2.3. Stream Response¶

A wrapper object used in streaming operations to encapsulate different types of response data.

Field	Type	Required	Description
`task`	`Task`		Optional (OneOf) A Task object containing the current state of the task.
`message`	`Message`		Optional (OneOf) A Message object containing a message from the agent.
`statusUpdate`	`TaskStatusUpdateEvent`		Optional (OneOf) An event indicating a task status update.
`artifactUpdate`	`TaskArtifactUpdateEvent`		Optional (OneOf) An event indicating a task artifact update.

Note: A `StreamResponse` MUST contain exactly one of the following: `task`, `message`, `statusUpdate`, `artifactUpdate`

This wrapper allows streaming endpoints to return different types of updates through a single response stream while maintaining type safety.

3.2.4. History Length Semantics¶

The `historyLength` parameter appears in multiple operations and controls how much task history is returned in responses. This parameter follows consistent semantics across all operations:

- **Unset/undefined**: No limit imposed; server returns its default amount of history (implementation-defined, may be all history)
- **0**: No history should be returned; the `history` field SHOULD be omitted
- **> 0**: Return at most this many recent messages from the task's history

3.2.5. Metadata¶

A flexible key-value map for passing additional context or parameters with operations. Metadata keys and are strings and values can be any valid value that can be represented in JSON. `Extensions` can be used to strongly type metadata values for specific use cases.

3.2.6 Service Parameters¶

A key-value map for passing horizontally applicable context or parameters with case-insensitive string keys and case-sensitive string values. The transmission mechanism for these service parameter key-value pairs is defined by the specific protocol binding (e.g., HTTP headers for HTTP-based bindings, gRPC metadata for gRPC bindings). Custom protocol bindings **MUST** specify how service parameters are transmitted in their binding specification.

Standard A2A Service Parameters:

Name	Description	Example Value
`A2A-Extensions`	Comma-separated list of extension URIs that the client wants to use for the request	`https://example.com/extensions/geolocation/v1,https://standards.org/extensions/citations/v1`
`A2A-Version`	The A2A protocol version that the client is using. If the version is not supported, the agent returns `VersionNotSupportedError`	`0.3`

As service parameter names MAY need to co-exist with other parameters defined by the underlying transport protocol or infrastructure, all service parameters defined by this specification will be prefixed with `a2a-`.

3.3. Operation Semantics¶

3.3.1. Idempotency¶

- **Get operations** (Get Task, List Tasks, Get Extended Agent Card) are naturally idempotent
- **Send Message** operations MAY be idempotent. Agents may utilize the messageId to detect duplicate messages.
- **Cancel Task** operations are idempotent - multiple cancellation requests have the same effect. A duplicate cancellation request MAY return `TaskNotFoundError` if the task has already been canceled and purged.

3.3.2. Error Handling¶

All operations may return errors in the following categories. Servers **MUST** return appropriate errors and **SHOULD** provide actionable information to help clients resolve issues.

Error Categories and Server Requirements:

- **Authentication Errors**: Invalid or missing credentials
  - Servers **MUST** reject requests with invalid or missing authentication credentials
  - Servers **SHOULD** include authentication challenge information in the error response
  - Servers **SHOULD** specify which authentication scheme is required
  - Example error codes: HTTP `401 Unauthorized`, gRPC `UNAUTHENTICATED`, JSON-RPC custom error
  - Example scenarios: Missing bearer token, expired API key, invalid OAuth token
- **Authorization Errors**: Insufficient permissions for requested operation
  - Servers **MUST** return an authorization error when the authenticated client lacks required permissions
  - Servers **SHOULD** indicate what permission or scope is missing (without leaking sensitive information about resources the client cannot access)
  - Servers **MUST NOT** reveal the existence of resources the client is not authorized to access
  - Example error codes: HTTP `403 Forbidden`, gRPC `PERMISSION_DENIED`, JSON-RPC custom error
  - Example scenarios: Attempting to access a task created by another user, insufficient OAuth scopes
- **Validation Errors**: Invalid input parameters or message format
  - Servers **MUST** validate all input parameters before processing
  - Servers **SHOULD** specify which parameter(s) failed validation and why
  - Servers **SHOULD** provide guidance on valid parameter values or formats

- Example error codes: HTTP `400 Bad Request`, gRPC `INVALID_ARGUMENT`, JSON-RPC `-32602 Invalid params`
- Example scenarios: Invalid task ID format, missing required message parts, unsupported content type

- **Resource Errors**: Requested task not found or not accessible
  - Servers **MUST** return a not found error when a requested resource does not exist or is not accessible to the authenticated client
  - Servers **SHOULD NOT** distinguish between "does not exist" and "not authorized" to prevent information leakage
  - Example error codes: HTTP `404 Not Found`, gRPC `NOT_FOUND`, JSON-RPC custom error (see A2A-specific errors)
  - Example scenarios: Task ID does not exist, task has been deleted, configuration not found

- **System Errors**: Internal agent failures or temporary unavailability
  - Servers **SHOULD** return appropriate error codes for temporary failures vs. permanent errors
  - Servers **MAY** include retry guidance (e.g., Retry-After header in HTTP)
  - Servers **SHOULD** log system errors for diagnostic purposes
  - Example error codes: HTTP `500 Internal Server Error` or `503 Service Unavailable`, gRPC `INTERNAL` or `UNAVAILABLE`, JSON-RPC `-32603 Internal error`
  - Example scenarios: Database connection failure, downstream service timeout, rate limit exceeded

Error Payload Structure:

All error responses in the A2A protocol, regardless of binding, **MUST** convey the following information:

1. **Error Code**: A machine-readable identifier for the error type (e.g., string code, numeric code, or protocol-specific status)
2. **Error Message**: A human-readable description of the error
3. **Error Details** (optional): An array of objects providing additional structured information about the error. Each object in the array **MUST** include a `@type` key that identifies the object's type (using ProtoJSON `Any` representation). Well-known types from the `google.rpc` error model (e.g., `ErrorInfo`, `BadRequest`) **SHOULD** be used where applicable. Error details may be used for:
   - Affected fields or parameters
   - Contextual information (e.g., task ID, timestamp)
   - Suggestions for resolution

Protocol bindings **MUST** map these elements to their native error representations while preserving semantic meaning. See binding-specific sections for concrete error format examples: JSON-RPC Error Handling, gRPC Error Handling, and HTTP/REST Error Handling.

A2A-Specific Errors:

Error Name	Description
`TaskNotFoundError`	The specified task ID does not correspond to an existing or accessible task. It might be invalid, expired, or already completed and purged.
`TaskNotCancelableError`	An attempt was made to cancel a task that is not in a cancelable state (e.g., it has already reached a terminal state like `TASK_STATE_COMPLETED`, `TASK_STATE_FAILED`, `TASK_STATE_CANCELED`, `TASK_STATE_REJECTED`).
`PushNotificationNotSupportedError`	Client attempted to use push notification features but the server agent does not support them (i.e., `AgentCard.capabilities.pushNotifications` is `false`).
`UnsupportedOperationError`	The requested operation or a specific aspect of it is not supported by this server agent implementation.
`ContentTypeNotSupportedError`	A Media Type provided in the request's message parts or implied for an artifact is not supported by the agent or the specific skill being invoked.
`InvalidAgentResponseError`	An agent returned a response that does not conform to the specification for the current method.
`ExtendedAgentCardNotConfiguredError`	The agent does not have an extended agent card configured when one is required for the requested operation.
`ExtensionSupportRequiredError`	Server requested use of an extension marked as `required: true` in the Agent Card but the client did not declare support for it in the request.
`VersionNotSupportedError`	The A2A protocol version specified in the request (via `A2A-Version` service parameter) is not supported by the agent.

3.3.3. Asynchronous Processing¶

A2A operations are designed for asynchronous task execution. Operations return immediately with either `Task` objects or `Message` objects, and when a Task is returned, processing continues in the background. Clients retrieve task updates through polling, streaming, or push notifications (see Section 3.5). Agents MAY accept additional messages for tasks in non-terminal states to enable multi-turn interactions (see Section 3.4).

3.3.4. Capability Validation¶

Agents declare optional capabilities in their `AgentCard`. When clients attempt to use operations or features that require capabilities not declared as supported in the Agent Card, the agent **MUST** return an appropriate error response:

- **Push Notifications**: If `AgentCard.capabilities.pushNotifications` is `false` or not present, operations related to push notification configuration (Create, Get, List, Delete) **MUST** return `PushNotificationNotSupportedError`.
- **Streaming**: If `AgentCard.capabilities.streaming` is `false` or not present, attempts to use `SendStreamingMessage` or `SubscribeToTask` operations **MUST** return `UnsupportedOperationError`.
- **Extended Agent Card**: If `AgentCard.capabilities.extendedAgentCard` is `false` or not present, attempts to call the Get Extended Agent Card operation **MUST** return `UnsupportedOperationError`. If the agent declares support but has not configured an extended card, it **MUST** return `ExtendedAgentCardNotConfiguredError`.
- **Extensions**: When a server requests use of an extension marked as `required: true` in the Agent Card but the client does not declare support for it, the agent **MUST** return `ExtensionSupportRequiredError`.

Clients **SHOULD** validate capability support by examining the Agent Card before attempting operations that require optional capabilities.

3.4. Multi-Turn Interactions¶

The A2A protocol supports multi-turn conversations through context identifiers and task references, enabling agents to maintain conversational continuity across multiple interactions.

3.4.1. Context Identifier Semantics¶

A `contextId` is an identifier that logically groups multiple related `Task` and `Message` objects, providing continuity across a series of interactions.

Generation and Assignment:

- Agents **MAY** generate a new `contextId` when processing a `Message` that does not include a `contextId` field
- If an agent generates a new `contextId`, it **MUST** be included in the response (either `Task` or `Message`)
- Agents **MAY** accept and preserve client-provided `contextId` values
- If an agent cannot accept a client-provided `contextId`, it **MUST** reject the request with an error and **MUST NOT** generate a new `contextId` for the response
- Clients **SHOULD NOT** provide a client-generated `contextId` to a server unless they understand how the server will process that `contextId`
- Server-generated `contextId` values **SHOULD** be treated as opaque identifiers by clients

Grouping and Scope:

- A `contextId` logically groups multiple `Task` objects and `Message` objects that are part of the same conversational context
- All tasks and messages with the same `contextId` **SHOULD** be treated as part of the same conversational session
- Agents **MAY** use the `contextId` to maintain internal state, conversational history, or LLM context across multiple interactions
- Agents **MAY** implement context expiration or cleanup policies and **SHOULD** document any such policies

3.4.2. Task Identifier Semantics¶

A `taskId` is a unique identifier for a `Task` object, representing a stateful unit of work with a defined lifecycle.

Generation and Assignment:

- Task IDs are **server-generated** when a new task is created in response to a `Message`
- Agents **MUST** generate a unique `taskId` for each new task they create
- The generated `taskId` **MUST** be included in the `Task` object returned to the client
- When a client includes a `taskId` in a `Message`, it **MUST** reference an existing task
- Agents **MUST** return a `TaskNotFoundError` if the provided `taskId` does not correspond to an existing task
- Client-provided `taskId` values for creating new tasks is **NOT** supported

3.4.3. Multi-Turn Conversation Patterns¶

The A2A protocol supports several patterns for multi-turn interactions:

Context Continuity:

- `Task` objects maintain conversation context through the `contextId` field
- Clients **MAY** include the `contextId` in subsequent messages to indicate continuation of a previous interaction
- Clients **MAY** use `taskId` (with or without `contextId`) to continue or refine a specific task
- Clients **MAY** use `contextId` without `taskId` to start a new task within an existing conversation context
- Agents **MUST** infer `contextId` from the task if only `taskId` is provided
- Agents **MUST** reject messages containing mismatching `contextId` and `taskId` (i.e., the provided `contextId` is different from that of the referenced `Task`).

Input Required State:

- Agents can request additional input mid-processing by transitioning a task to the `input-required` state
- The client continues the interaction by sending a new message with the same `taskId` and `contextId`

Follow-up Messages:

- Clients can send additional messages with `taskId` references to continue or refine existing tasks
- Clients **SHOULD** use the `referenceTaskIds` field in `Message` to explicitly reference related tasks
- Agents **SHOULD** use referenced tasks to understand the context and intent of follow-up requests

Context Inheritance:

- New tasks created within the same `contextId` can inherit context from previous interactions
- Agents **SHOULD** leverage the shared `contextId` to provide contextually relevant responses

3.5. Task Update Delivery Mechanisms¶

The A2A protocol provides three complementary mechanisms for clients to receive updates about task progress and completion.

3.5.1. Overview of Update Mechanisms¶

Polling (Get Task):

- Client periodically calls Get Task (Section 3.1.3) to check task status
- Simple to implement, works with all protocol bindings
- Higher latency, potential for unnecessary requests
- Best for: Simple integrations, infrequent updates, clients behind restrictive firewalls

Streaming:

- Real-time delivery of events as they occur
- Operations: Send Streaming Message (Section 3.1.2) and Subscribe to Task (Section 3.1.6)
- Low latency, efficient for frequent updates
- Requires persistent connection support
- Best for: Interactive applications, real-time dashboards, live progress monitoring

- Requires `AgentCard.capabilities.streaming` to be `true`

Push Notifications (WebHooks):

- Agent sends HTTP POST requests to client-registered endpoints when task state changes
- Client does not maintain persistent connection
- Asynchronous delivery, client must be reachable via HTTP
- Best for: Server-to-server integrations, long-running tasks, event-driven architectures
- Operations: Create (Section 3.1.7), Get (Section 3.1.8), List (Section 3.1.9), Delete (Section 3.1.10)
- Event types: TaskStatusUpdateEvent (Section 4.2.1), TaskArtifactUpdateEvent (Section 4.2.2), WebHook payloads (Section 4.3)
- Requires `AgentCard.capabilities.pushNotifications` to be `true`
- Regardless of the protocol binding being used by the agent, WebHook calls use plain HTTP and the JSON payloads as defined in the HTTP protocol binding

3.5.2. Streaming Event Delivery¶

Event Ordering:

All implementations MUST deliver events in the order they were generated. Events MUST NOT be reordered during transmission, regardless of protocol binding.

Multiple Streams Per Task:

An agent MAY serve multiple concurrent streams to one or more clients for the same task. This allows multiple clients (or the same client with multiple connections) to independently subscribe to and receive updates about a task's progress.

When multiple streams are active for a task:

- Events MUST be broadcast to all active streams for that task
- Each stream MUST receive the same events in the same order
- Closing one stream MUST NOT affect other active streams for the same task
- The task lifecycle is independent of any individual stream's lifecycle

This capability enables scenarios such as:

- Multiple team members monitoring the same long-running task
- A client reconnecting to a task after a network interruption by opening a new stream
- Different applications or dashboards displaying real-time updates for the same task

3.5.3. Push Notification Delivery¶

Push notifications are delivered via HTTP POST to client-registered webhook endpoints. The delivery semantics and reliability guarantees are defined in Section 4.3.

3.6 Versioning¶

The specific version of the A2A protocol in use is identified using the `Major.Minor` elements (e.g. `1.0`) of the corresponding A2A specification version. Patch version numbers used by the specification, do not affect protocol compatibility. Patch version numbers SHOULD NOT be used in requests, responses and Agent Cards, and MUST not be considered when clients and servers negotiate protocol versions.

3.6.1 Client Responsibilities¶

Clients MUST send the `A2A-Version` header with each request to maintain compatibility after an agent upgrades to a new version of the protocol (except for 0.3 Clients - 0.3 will be assumed for empty header). Sending the `A2A-Version` header also provides visibility to agents about version usage in the ecosystem, which can help inform the risks of inplace version upgrades.

Example of HTTP GET Request with Version Header:

```
GET /tasks/task-123 HTTP/1.1
Host: agent.example.com
A2A-Version: 1.0
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Accept: application/json
```

Clients MAY provide the `A2A-Version` as a request parameter instead of a header.

Example of HTTP GET Request with Version request parameter:

```
GET /tasks/task-123?A2A-Version=1.0 HTTP/1.1
Host: agent.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Accept: application/json
```

3.6.2 Server Responsibilities¶

Agents MUST process requests using the semantics of the requested `A2A-Version` (matching `Major.Minor`). If the version is not supported by the interface, agents MUST return a `VersionNotSupportedError`.

Agents MUST interpret empty value as 0.3 version.

Agents CAN expose multiple interfaces for the same transport with different versions under the same or different URLs.

3.6.3 Tooling support¶

Tooling libraries and SDKs that implement the A2A protocol MUST provide mechanisms to help clients manage protocol versioning, such as negotiation of the transport and protocol version used. Client Agents that require the latest features of the protocol should be configured to request specific versions and avoid automatic fallback to older versions, to prevent silently losing functionality.

3.7 Messages and Artifacts¶

Messages and Artifacts serve distinct purposes within the A2A protocol. The core interaction model defined by A2A is for clients to send messages to initiate a task that produces one or more artifacts.

Messages play several key roles:

- **Task Initiation**: Clients send Messages to agents to initiate new tasks.
- **Clarification Messages**: Agents may send Messages back to the client to request clarification prior to initiating a task.
- **Status Messages**: Agents attach Messages to status update events to inform clients about task progress, request additional input, or provide informational updates.
- **Task Interaction**: Clients send Messages to provide additional input or instructions for ongoing tasks.

Messages SHOULD NOT be used to deliver task outputs. Results SHOULD BE returned using Artifacts associated with a Task. This separation allows for a clear distinction between communication (Messages) and data output (Artifacts).

The Task History field contains Messages exchanged during task execution. However, not all Messages are guaranteed to be persisted in the Task history; for example, transient informational messages may not be stored. Messages exchanged prior to task creation may not be stored in Task history. The agent is responsible to determine which Messages are persisted in the Task History.

Clients using streaming to retrieve task updates MAY not receive all status update messages if the client is disconnected and then reconnects. Messages MUST NOT be considered a reliable delivery mechanism for critical information.

Agents MAY choose to persist all Messages that contain important information in the Task history to ensure clients can retrieve it later. However, clients MUST NOT rely on this behavior unless negotiated out-of-band.

4. Protocol Data Model¶

The A2A protocol defines a canonical data model using Protocol Buffers. All protocol bindings **MUST** provide functionally equivalent representations of these data structures.

4.1. Core Objects¶

4.1.1. Task¶

`Task` is the core unit of action for A2A. It has a current status and when results are created for the task they are stored in the artifact. If there are multiple turns for a task, these are stored in history.

Field	Type	Required	Description
`id`	`string`	Yes	Unique identifier (e.g. UUID) for the task, generated by the server for a new task.
`contextId`	`string`	No	Unique identifier (e.g. UUID) for the contextual collection of interactions (tasks and messages).
`status`	`TaskStatus`	Yes	The current status of a `Task`, including `state` and a `message`.
`artifacts`	array of `Artifact`	No	A set of output artifacts for a `Task`.
`history`	array of `Message`	No	The history of interactions from a `Task`.
`metadata`	`object`	No	A key/value object to store custom metadata about a task.

4.1.2. TaskStatus¶

A container for the status of a task

Field	Type	Required	Description
`state`	`TaskState`	Yes	The current state of this task.
`message`	`Message`	No	A message associated with the status.
`timestamp`	`timestamp`	No	ISO 8601 Timestamp when the status was recorded. Example: "2023-10-27T10:00:00Z"

4.1.3. TaskState¶

Defines the possible lifecycle states of a `Task`.

Value	Description
`TASK_STATE_UNSPECIFIED`	The task is in an unknown or indeterminate state.
`TASK_STATE_SUBMITTED`	Indicates that a task has been successfully submitted and acknowledged.
`TASK_STATE_WORKING`	Indicates that a task is actively being processed by the agent.
`TASK_STATE_COMPLETED`	Indicates that a task has finished successfully. This is a terminal state.
`TASK_STATE_FAILED`	Indicates that a task has finished with an error. This is a terminal state.
`TASK_STATE_CANCELED`	Indicates that a task was canceled before completion. This is a terminal state.
`TASK_STATE_INPUT_REQUIRED`	Indicates that the agent requires additional user input to proceed. This is an interrupted state.
`TASK_STATE_REJECTED`	Indicates that the agent has decided to not perform the task. This may be done during initial task creation or later once an agent has determined it can't or won't proceed. This is a terminal state.
`TASK_STATE_AUTH_REQUIRED`	Indicates that authentication is required to proceed. This is an interrupted state.

4.1.4. Message¶

`Message` is one unit of communication between client and server. It can be associated with a context and/or a task. For server messages, `context_id` must be provided, and `task_id` only if a task was created. For client messages, both fields are optional, with the caveat that if both are provided, they have to match (the `context_id` has to be the one that is set on the task). If only `task_id` is provided, the server will infer `context_id` from it.

Field	Type	Required	Description
`messageId`	`string`	Yes	The unique identifier (e.g. UUID) of the message. This is created by the message creator.
`contextId`	`string`	No	Optional. The context id of the message. If set, the message will be associated with the given context.
`taskId`	`string`	No	Optional. The task id of the message. If set, the message will be associated with the given task.
`role`	`Role`	Yes	Identifies the sender of the message.
`parts`	array of `Part`	Yes	Parts is the container of the message content.
`metadata`	`object`	No	Optional. Any metadata to provide along with the message.
`extensions`	array of `string`	No	The URIs of extensions that are present or contributed to this Message.
`referenceTaskIds`	array of `string`	No	A list of task IDs that this message references for additional context.

4.1.5. Role¶

Defines the sender of a message in A2A protocol communication.

Value	Description
`ROLE_UNSPECIFIED`	The role is unspecified.
`ROLE_USER`	The message is from the client to the server.
`ROLE_AGENT`	The message is from the server to the client.

4.1.6. Part¶

`Part` represents a container for a section of communication content. Parts can be purely textual, some sort of file (image, video, etc) or a structured data blob (i.e. JSON).

Field	Type	Required	Description
`text`	`string`	Optional (OneOf)	The string content of the `text` part.
`raw`	`bytes`	Optional (OneOf)	The `raw` byte content of a file. In JSON serialization, this is encoded as a base64 string.
`url`	`string`	Optional (OneOf)	A `url` pointing to the file's content.
`data`	`any`	Optional (OneOf)	Arbitrary structured `data` as a JSON value (object, array, string, number, boolean, or null).
`metadata`	`object`	No	Optional. metadata associated with this part.
`filename`	`string`	No	An optional `filename` for the file (e.g., "document.pdf").
`mediaType`	`string`	No	The `media_type` (MIME type) of the part content (e.g., "text/plain", "application/json", "image/png"). This field is available for all part types.

Note: A `Part` MUST contain exactly one of the following: `text`, `raw`, `url`, `data`

4.1.7. Artifact¶

Artifacts represent task outputs.

Field	Type	Required	Description
`artifactId`	`string`	Yes	Unique identifier (e.g. UUID) for the artifact. It must be unique within a task.
`name`	`string`	No	A human readable name for the artifact.
`description`	`string`	No	Optional. A human readable description of the artifact.
`parts`	array of `Part`	Yes	The content of the artifact. Must contain at least one part.
`metadata`	`object`	No	Optional. Metadata included with the artifact.
`extensions`	array of `string`	No	The URIs of extensions that are present or contributed to this Artifact.

4.2. Streaming Events¶

4.2.1. TaskStatusUpdateEvent¶

An event sent by the agent to notify the client of a change in a task's status.

Field	Type	Required	Description
`taskId`	`string`	Yes	The ID of the task that has changed.
`contextId`	`string`	Yes	The ID of the context that the task belongs to.
`status`	`TaskStatus`	Yes	The new status of the task.
`metadata`	`object`	No	Optional. Metadata associated with the task update.

4.2.2. TaskArtifactUpdateEvent¶

A task delta where an artifact has been generated.

Field	Type	Required	Description
`taskId`	`string`	Yes	The ID of the task for this artifact.
`contextId`	`string`	Yes	The ID of the context that this task belongs to.
`artifact`	`Artifact`	Yes	The artifact that was generated or updated.
`append`	`boolean`	No	If true, the content of this artifact should be appended to a previously sent artifact with the same ID.
`lastChunk`	`boolean`	No	If true, this is the final chunk of the artifact.
`metadata`	`object`	No	Optional. Metadata associated with the artifact update.

4.3. Push Notification Objects¶

4.3.1. PushNotificationConfig¶

Error: Message `PushNotificationConfig` not found.

4.3.2. AuthenticationInfo¶

Defines authentication details, used for push notifications.

Field	Type	Required	Description
`scheme`	`string`	Yes	HTTP Authentication Scheme from the IANA registry. Examples: `Bearer`, `Basic`, `Digest`. Scheme names are case-insensitive per RFC 9110 Section 11.1.
`credentials`	`string`	No	Push Notification credentials. Format depends on the scheme (e.g., token for Bearer).

4.3.3. Push Notification Payload¶

When a task update occurs, the agent sends an HTTP POST request to the configured webhook URL. The payload uses the same `StreamResponse` format as streaming operations, allowing push notifications to deliver the same event types as real-time streams.

Request Format:

```
POST {webhook_url}
Authorization: {authentication_scheme} {credentials}
Content-Type: application/a2a+json

{
  /* StreamResponse object - one of: */
  "task": { /* Task object */ },
  "message": { /* Message object */ },
  "statusUpdate": { /* TaskStatusUpdateEvent object */ },
  "artifactUpdate": { /* TaskArtifactUpdateEvent object */ }
}
```

Payload Structure:

The webhook payload is a `StreamResponse` object containing exactly one of the following:

- **task**: A `Task` object with the current task state
- **message**: A `Message` object containing a message response
- **statusUpdate**: A `TaskStatusUpdateEvent` indicating a status change
- **artifactUpdate**: A `TaskArtifactUpdateEvent` indicating artifact updates

Authentication:

The agent MUST include authentication credentials in the request headers as specified in the `PushNotificationConfig.authentication` field. The format follows standard HTTP authentication patterns (Bearer tokens, Basic auth, etc.).

Client Responsibilities:

- Clients MUST respond with HTTP 2xx status codes to acknowledge successful receipt
- Clients SHOULD process notifications idempotently, as duplicate deliveries may occur
- Clients MUST validate the task ID matches an expected task
- Clients SHOULD implement appropriate security measures to verify the notification source

Server Guarantees:

- Agents MUST attempt delivery at least once for each configured webhook
- Agents MAY implement retry logic with exponential backoff for failed deliveries
- Agents SHOULD include a reasonable timeout for webhook requests (recommended: 10-30 seconds)
- Agents MAY stop attempting delivery after a configured number of consecutive failures

For detailed security guidance on push notifications, see Section 13.2 Push Notification Security.

4.4. Agent Discovery Objects¶

4.4.1. AgentCard¶

A self-describing manifest for an agent. It provides essential metadata including the agent's identity, capabilities, skills, supported communication methods, and security requirements.

Field	Type	Required	Description
`name`	`string`	Yes	A human readable name for the agent. Example: "Recipe Agent"
`description`	`string`	Yes	A human-readable description of the agent, assisting users and other agents in understanding its purpose. Example: "Agent that helps users with recipes and cooking."
`supportedInterfaces`	array of `AgentInterface`	Yes	Ordered list of supported interfaces. The first entry is preferred.
`provider`	`AgentProvider`	No	The service provider of the agent.
`version`	`string`	Yes	The version of the agent. Example: "1.0.0"
`documentationUrl`	`string`	No	A URL providing additional documentation about the agent.
`capabilities`	`AgentCapabilities`	Yes	A2A Capability set supported by the agent.
`securitySchemes`	map of string to `SecurityScheme`	No	The security scheme details used for authenticating with this agent.
`securityRequirements`	array of `SecurityRequirement`	No	Security requirements for contacting the agent.
`defaultInputModes`	array of `string`	Yes	The set of interaction modes that the agent supports across all skills. This can be overridden per skill. Defined as media types.
`defaultOutputModes`	array of `string`	Yes	The media types supported as outputs from this agent.
`skills`	array of `AgentSkill`	Yes	Skills represent the abilities of an agent. It is largely a descriptive concept but represents a more focused set of behaviors that the agent is likely to succeed at.
`signatures`	array of `AgentCardSignature`	No	JSON Web Signatures computed for this `AgentCard`.
`iconUrl`	`string`	No	Optional. A URL to an icon for the agent.

4.4.2. AgentProvider¶

Represents the service provider of an agent.

Field	Type	Required	Description
`url`	`string`	Yes	A URL for the agent provider's website or relevant documentation. Example: "https://ai.google.dev"
`organization`	`string`	Yes	The name of the agent provider's organization. Example: "Google"

4.4.3. AgentCapabilities¶

Defines optional capabilities supported by an agent.

Field	Type	Required	Description
`streaming`	`boolean`	No	Indicates if the agent supports streaming responses.
`pushNotifications`	`boolean`	No	Indicates if the agent supports sending push notifications for asynchronous task updates.
`extensions`	array of `AgentExtension`	No	A list of protocol extensions supported by the agent.
`extendedAgentCard`	`boolean`	No	Indicates if the agent supports providing an extended agent card when authenticated.

4.4.4. AgentExtension¶

A declaration of a protocol extension supported by an Agent.

Field	Type	Required	Description
`uri`	`string`	No	The unique URI identifying the extension.
`description`	`string`	No	A human-readable description of how this agent uses the extension.
`required`	`boolean`	No	If true, the client must understand and comply with the extension's requirements.
`params`	`object`	No	Optional. Extension-specific configuration parameters.

4.4.5. AgentSkill¶

Represents a distinct capability or function that an agent can perform.

Field	Type	Required	Description
`id`	`string`	Yes	A unique identifier for the agent's skill.
`name`	`string`	Yes	A human-readable name for the skill.
`description`	`string`	Yes	A detailed description of the skill.
`tags`	array of `string`	Yes	A set of keywords describing the skill's capabilities.
`examples`	array of `string`	No	Example prompts or scenarios that this skill can handle.
`inputModes`	array of `string`	No	The set of supported input media types for this skill, overriding the agent's defaults.
`outputModes`	array of `string`	No	The set of supported output media types for this skill, overriding the agent's defaults.
`securityRequirements`	array of `SecurityRequirement`	No	Security schemes necessary for this skill.

4.4.6. AgentInterface¶

Declares a combination of a target URL, transport and protocol version for interacting with the agent. This allows agents to expose the same functionality over multiple protocol binding mechanisms.

Field	Type	Required	Description
`url`	`string`	Yes	The URL where this interface is available. Must be a valid absolute HTTPS URL in production. Example: "https://api.example.com/a2a/v1", "https://grpc.example.com/a2a"
`protocolBinding`	`string`	Yes	The protocol binding supported at this URL. This is an open form string, to be easily extended for other protocol bindings. The core ones officially supported are `JSONRPC`, `GRPC` and `HTTP+JSON`.
`tenant`	`string`	No	Optional. An opaque string used for routing requests to a specific agent or tenant when multiple agents are served behind a single A2A endpoint. When set, clients MUST include this value in the `tenant` field of all request messages sent to this interface. The server is responsible for interpreting the value and routing requests accordingly; the protocol does not define its format or semantics.
`protocolVersion`	`string`	Yes	The version of the A2A protocol this interface exposes. Use the latest supported minor version per major version. Examples: "0.3", "1.0"

4.4.7. AgentCardSignature¶

AgentCardSignature represents a JWS signature of an AgentCard. This follows the JSON format of an RFC 7515 JSON Web Signature (JWS).

Field	Type	Required	Description
`protected`	`string`	Yes	Required. The protected JWS header for the signature. This is always a base64url-encoded JSON object.
`signature`	`string`	Yes	Required. The computed signature, base64url-encoded.
`header`	`object`	No	The unprotected JWS header values.

4.5. Security Objects¶

4.5.1. SecurityScheme¶

Defines a security scheme that can be used to secure an agent's endpoints. This is a discriminated union type based on the OpenAPI 3.2 Security Scheme Object. See: https://spec.openapis.org/oas/v3.2.0.html#security-scheme-object

Field	Type	Required	Description
`apiKeySecurityScheme`	`APIKeySecurityScheme`	Optional (OneOf)	API key-based authentication.
`httpAuthSecurityScheme`	`HTTPAuthSecurityScheme`	Optional (OneOf)	HTTP authentication (Basic, Bearer, etc.).
`oauth2SecurityScheme`	`OAuth2SecurityScheme`	Optional (OneOf)	OAuth 2.0 authentication.
`openIdConnectSecurityScheme`	`OpenIdConnectSecurityScheme`	Optional (OneOf)	OpenID Connect authentication.
`mtlsSecurityScheme`	`MutualTlsSecurityScheme`	Optional (OneOf)	Mutual TLS authentication.

Note: A `SecurityScheme` MUST contain exactly one of the following: `apiKeySecurityScheme`, `httpAuthSecurityScheme`, `oauth2SecurityScheme`, `openIdConnectSecurityScheme`, `mtlsSecurityScheme`

4.5.2. APIKeySecurityScheme¶

Defines a security scheme using an API key.

Field	Type	Required	Description
`description`	`string`	No	An optional description for the security scheme.
`location`	`string`	Yes	The location of the API key. Valid values are "query", "header", or "cookie".
`name`	`string`	Yes	The name of the header, query, or cookie parameter to be used.

4.5.3. HTTPAuthSecurityScheme¶

Defines a security scheme using HTTP authentication.

Field	Type	Required	Description
`description`	`string`	No	An optional description for the security scheme.
`scheme`	`string`	Yes	The name of the HTTP Authentication scheme to be used in the Authorization header, as defined in RFC7235 (e.g., "Bearer"). This value should be registered in the IANA Authentication Scheme registry.
`bearerFormat`	`string`	No	A hint to the client to identify how the bearer token is formatted (e.g., "JWT"). Primarily for documentation purposes.

4.5.4. OAuth2SecurityScheme¶

Defines a security scheme using OAuth 2.0.

Field	Type	Required	Description
`description`	`string`	No	An optional description for the security scheme.
`flows`	`OAuthFlows`	Yes	An object containing configuration information for the supported OAuth 2.0 flows.
`oauth2MetadataUrl`	`string`	No	URL to the OAuth2 authorization server metadata RFC 8414. TLS is required.

4.5.5. OpenIdConnectSecurityScheme¶

Defines a security scheme using OpenID Connect.

Field	Type	Required	Description
`description`	`string`	No	An optional description for the security scheme.
`openIdConnectUrl`	`string`	Yes	The OpenID Connect Discovery URL for the OIDC provider's metadata.

4.5.6. MutualTlsSecurityScheme¶

Defines a security scheme using mTLS authentication.

Field	Type	Required	Description
`description`	`string`	No	An optional description for the security scheme.

4.5.7. OAuthFlows¶

Defines the configuration for the supported OAuth 2.0 flows.

Field	Type	Required	Description
`authorizationCode`	`AuthorizationCodeOAuthFlow`	Optional (OneOf)	Configuration for the OAuth Authorization Code flow.
`clientCredentials`	`ClientCredentialsOAuthFlow`	Optional (OneOf)	Configuration for the OAuth Client Credentials flow.
`implicit`	`ImplicitOAuthFlow`	Optional (OneOf)	Deprecated: Use Authorization Code + PKCE instead.
`password`	`PasswordOAuthFlow`	Optional (OneOf)	Deprecated: Use Authorization Code + PKCE or Device Code.
`deviceCode`	`DeviceCodeOAuthFlow`	Optional (OneOf)	Configuration for the OAuth Device Code flow.

Note: A `OAuthFlows` MUST contain exactly one of the following: `authorizationCode`, `clientCredentials`, `implicit`, `password`, `deviceCode`

4.5.8. AuthorizationCodeOAuthFlow¶

Defines configuration details for the OAuth 2.0 Authorization Code flow.

Field	Type	Required	Description
`authorizationUrl`	`string`	Yes	The authorization URL to be used for this flow.
`tokenUrl`	`string`	Yes	The token URL to be used for this flow.
`refreshUrl`	`string`	No	The URL to be used for obtaining refresh tokens.
`scopes`	map of string to `string`	Yes	The available scopes for the OAuth2 security scheme.
`pkceRequired`	`boolean`	No	Indicates if PKCE (RFC 7636) is required for this flow. PKCE should always be used for public clients and is recommended for all clients.

4.5.9. ClientCredentialsOAuthFlow¶

Defines configuration details for the OAuth 2.0 Client Credentials flow.

Field	Type	Required	Description
`tokenUrl`	`string`	Yes	The token URL to be used for this flow.
`refreshUrl`	`string`	No	The URL to be used for obtaining refresh tokens.
`scopes`	map of string to `string`	Yes	The available scopes for the OAuth2 security scheme.

4.5.10. DeviceCodeOAuthFlow¶

Defines configuration details for the OAuth 2.0 Device Code flow (RFC 8628). This flow is designed for input-constrained devices such as IoT devices, and CLI tools where the user authenticates on a separate device.

Field	Type	Required	Description
`deviceAuthorizationUrl`	`string`	Yes	The device authorization endpoint URL.
`tokenUrl`	`string`	Yes	The token URL to be used for this flow.
`refreshUrl`	`string`	No	The URL to be used for obtaining refresh tokens.
`scopes`	map of string to `string`	Yes	The available scopes for the OAuth2 security scheme.

4.6. Extensions¶

The A2A protocol supports extensions to provide additional functionality or data beyond the core specification while maintaining backward compatibility and interoperability. Extensions allow agents to declare additional capabilities such as protocol enhancements or vendor-specific features, maintain compatibility with clients that don't support specific extensions, enable innovation through experimental or domain-specific features without modifying the core protocol, and facilitate standardization by providing a pathway for community-developed features to become part of the core specification.

4.6.1. Extension Declaration¶

Agents declare their supported extensions in the `AgentCard` using the `extensions` field, which contains an array of `AgentExtension` objects.

Example: Agent declaring extension support in AgentCard:

```
{
  "name": "Research Assistant Agent",
  "description": "AI agent for academic research and fact-checking",
  "supportedInterfaces": [
    {
      "url": "https://research-agent.example.com/a2a/v1",
      "protocolBinding": "HTTP+JSON",
      "protocolVersion": "0.3",
    }
  ],
```

```
  "capabilities": {
    "streaming": false,
    "pushNotifications": false,
    "extensions": [
      {
        "uri": "https://standards.org/extensions/citations/v1",
        "description": "Provides citation formatting and source verification",
        "required": false
      },
      {
        "uri": "https://example.com/extensions/geolocation/v1",
        "description": "Location-based search capabilities",
        "required": false
      }
    ]
  },
  "defaultInputModes": ["text/plain"],
  "defaultOutputModes": ["text/plain"],
  "skills": [
    {
      "id": "academic-research",
      "name": "Academic Research Assistant",
      "description": "Provides research assistance with citations and source verification",
      "tags": ["research", "citations", "academic"],
      "examples": ["Find peer-reviewed articles on climate change"],
      "inputModes": ["text/plain"],
      "outputModes": ["text/plain"]
    }
  ]
}
```

Clients indicate their desire to opt into the use of specific extensions through binding-specific mechanisms such as HTTP headers, gRPC metadata, or JSON-RPC request parameters that identify the extension identifiers they wish to utilize during the interaction.

Example: HTTP client opting into extensions using headers:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token
A2A-Extensions: https://example.com/extensions/geolocation/v1,https://standards.org/extensions/citations/v1

{
  "message": {
    "role": "ROLE_USER",
    "parts": [{"text": "Find restaurants near me"}],
    "extensions": ["https://example.com/extensions/geolocation/v1"],
    "metadata": {
      "https://example.com/extensions/geolocation/v1": {
        "latitude": 37.7749,
        "longitude": -122.4194
      }
    }
  }
}
```

4.6.2. Extensions Points¶

Extensions can be integrated into the A2A protocol at several well-defined extension points:

Message Extensions:

Messages can be extended to allow clients to provide additional strongly typed context or parameters relevant to the message being sent, or TaskStatus Messages to include extra information about the task's progress.

Example: A location extension using the extensions and metadata arrays:

```
{
  "role": "ROLE_USER",
  "parts": [
    {"text": "Find restaurants near me"}
  ],
  "extensions": ["https://example.com/extensions/geolocation/v1"],
  "metadata": {
    "https://example.com/extensions/geolocation/v1": {
      "latitude": 37.7749,
      "longitude": -122.4194,
      "accuracy": 10.0,
      "timestamp": "2025-10-21T14:30:00Z"
    }
  }
}
```

Artifact Extensions:

Artifacts can include extension data to provide strongly typed context or metadata about the generated content.

Example: An artifact with citation extension for research sources:

```
{
  "artifactId": "research-summary-001",
  "name": "Climate Change Summary",
  "parts": [
    {
```

```
      "text": "Global temperatures have risen by 1.1°C since pre-industrial times, with significant impacts on weather patterns and sea levels."
    }
  ],
  "extensions": ["https://standards.org/extensions/citations/v1"],
  "metadata": {
    "https://standards.org/extensions/citations/v1": {
      "sources": [
        {
          "title": "Global Temperature Anomalies - 2023 Report",
          "authors": ["Smith, J.", "Johnson, M."],
          "url": "https://climate.gov/reports/2023-temperature",
          "accessDate": "2025-10-21",
          "relevantText": "Global temperatures have risen by 1.1°C"
        }
      ]
    }
  }
}
```

4.6.3. Extension Versioning and Compatibility¶

Extensions **SHOULD** include version information in their URI identifier. This allows clients and agents to negotiate compatible versions of extensions during interactions. A new URI **MUST** be created for breaking changes to an extension.

If a client requests a versions of an extension that the agent does not support, the agent **SHOULD** ignore the extension for that interaction and proceed without it, unless the extension is marked as `required` in the AgentCard, in which case the agent **MUST** return an error indicating unsupported extension. It **MUST NOT** fall back to a previous version of the extension automatically.

5. Protocol Binding Requirements and Interoperability¶

5.1. Functional Equivalence Requirements¶

When an agent supports multiple protocols, all supported protocols **MUST**:

- **Identical Functionality**: Provide the same set of operations and capabilities
- **Consistent Behavior**: Return semantically equivalent results for the same requests
- **Same Error Handling**: Map errors consistently using appropriate protocol-specific codes
- **Equivalent Authentication**: Support the same authentication schemes declared in the AgentCard

5.2. Protocol Selection and Negotiation¶

- **Agent Declaration**: Agents **MUST** declare all supported protocols in their AgentCard
- **Client Choice**: Clients **MAY** choose any protocol declared by the agent
- **Fallback Behavior**: Clients **SHOULD** implement fallback logic for alternative protocols

5.3. Method Mapping Reference¶

Functionality	JSON-RPC Method	gRPC Method	REST Endpoint
Send message	`SendMessage`	`SendMessage`	`POST /message:send`
Send streaming message	`SendStreamingMessage`	`SendStreamingMessage`	`POST /message:stream`
Get task	`GetTask`	`GetTask`	`GET /tasks/{id}`
List tasks	`ListTasks`	`ListTasks`	`GET /tasks`
Cancel task	`CancelTask`	`CancelTask`	`POST /tasks/{id}:cancel`
Subscribe to task	`SubscribeToTask`	`SubscribeToTask`	`POST /tasks/{id}:subscribe`
Create push notification config	`CreateTaskPushNotificationConfig`	`CreateTaskPushNotificationConfig`	`POST /tasks/{id}/pushNotificationConfigs`
Get push notification config	`GetTaskPushNotificationConfig`	`GetTaskPushNotificationConfig`	`GET /tasks/{id}/pushNotificationConfigs/{configId}`
List push notification configs	`ListTaskPushNotificationConfigs`	`ListTaskPushNotificationConfigs`	`GET /tasks/{id}/pushNotificationConfigs`
Delete push notification config	`DeleteTaskPushNotificationConfig`	`DeleteTaskPushNotificationConfig`	`DELETE /tasks/{id}/pushNotificationConfigs/{configId}`
Get extended Agent Card	`GetExtendedAgentCard`	`GetExtendedAgentCard`	`GET /extendedAgentCard`

5.4. Error Code Mappings¶

All A2A-specific errors defined in Section 3.3.2 **MUST** be mapped to binding-specific error representations. The following table provides the canonical mappings for each standard protocol binding:

A2A Error Type	JSON-RPC Code	gRPC Status	HTTP Status
`TaskNotFoundError`	`-32001`	`NOT_FOUND`	`404 Not Found`
`TaskNotCancelableError`	`-32002`	`FAILED_PRECONDITION`	`400 Bad Request`
`PushNotificationNotSupportedError`	`-32003`	`FAILED_PRECONDITION`	`400 Bad Request`
`UnsupportedOperationError`	`-32004`	`FAILED_PRECONDITION`	`400 Bad Request`
`ContentTypeNotSupportedError`	`-32005`	`INVALID_ARGUMENT`	`400 Bad Request`
`InvalidAgentResponseError`	`-32006`	`INTERNAL`	`500 Internal Server Error`
`ExtendedAgentCardNotConfiguredError`	`-32007`	`FAILED_PRECONDITION`	`400 Bad Request`
`ExtensionSupportRequiredError`	`-32008`	`FAILED_PRECONDITION`	`400 Bad Request`
`VersionNotSupportedError`	`-32009`	`FAILED_PRECONDITION`	`400 Bad Request`

Custom Binding Requirements:

Custom protocol bindings **MUST** define equivalent error code mappings that preserve the semantic meaning of each A2A error type. The binding specification **SHOULD** provide a similar mapping table showing how each A2A error type is represented in the custom binding's native error format.

For binding-specific error structures and examples, see:

- JSON-RPC Error Handling
- gRPC Error Handling
- HTTP/REST Error Handling

5.5. JSON Field Naming Convention¶

All JSON serializations of the A2A protocol data model **MUST** use **camelCase** naming for field names, not the snake_case convention used in Protocol Buffer definitions.

Naming Convention:

- Protocol Buffer field: `protocol_version` → JSON field: `protocolVersion`
- Protocol Buffer field: `context_id` → JSON field: `contextId`
- Protocol Buffer field: `default_input_modes` → JSON field: `defaultInputModes`
- Protocol Buffer field: `push_notification_config` → JSON field: `pushNotificationConfig`

Enum Values:

- Enum values **MUST** be represented according to the ProtoJSON specification, which serializes enums as their string names **as defined in the Protocol Buffer definition** (typically SCREAMING_SNAKE_CASE).

Examples:

- Protocol Buffer enum: `TASK_STATE_INPUT_REQUIRED` → JSON value: `"TASK_STATE_INPUT_REQUIRED"`
- Protocol Buffer enum: `ROLE_USER` → JSON value: `"ROLE_USER"`

Note: This follows the ProtoJSON specification as adopted in ADR-001.

5.6. Data Type Conventions¶

This section documents conventions for common data types used throughout the A2A protocol, particularly as they apply to protocol bindings.

5.6.1. Timestamps¶

The A2A protocol uses `google.protobuf.Timestamp` for all timestamp fields in the Protocol Buffer definitions. When serialized to JSON (in JSON-RPC, HTTP/REST, or other JSON-based bindings), these timestamps **MUST** be represented as ISO 8601 formatted strings in UTC timezone.

Format Requirements:

- **Format:** ISO 8601 combined date and time representation
- **Timezone:** UTC (denoted by 'Z' suffix)
- **Precision:** Millisecond precision **SHOULD** be used where available
- **Pattern:** `YYYY-MM-DDTHH:mm:ss.sssZ`

Examples:

```
{
  "timestamp": "2025-10-28T10:30:00.000Z",
  "createdAt": "2025-10-28T14:25:33.142Z",
  "lastModified": "2025-10-31T17:45:22.891Z"
}
```

Implementation Notes:

- Protocol Buffer's `google.protobuf.Timestamp` represents time as seconds since Unix epoch (January 1, 1970, 00:00:00 UTC) plus nanoseconds
- JSON serialization automatically converts this to ISO 8601 format when using standard Protocol Buffer JSON encoding
- Clients and servers **MUST** parse and generate ISO 8601 timestamps correctly
- When millisecond precision is not available, the fractional seconds portion **MAY** be omitted or zero-filled
- Timestamps **MUST NOT** include timezone offsets other than 'Z' (all times are UTC)

5.7. Field Presence and Optionality¶

The Protocol Buffer definition in `specification/a2a.proto` uses `google.api.field_behavior` annotations to indicate whether fields are `REQUIRED`. These annotations serve as both documentation and validation hints for implementations.

Required Fields:

Fields marked with `[(google.api.field_behavior) = REQUIRED]` indicate that the field **MUST** be present and set in valid messages. Implementations **SHOULD** validate these requirements and reject messages with missing required fields. Arrays marked as required **MUST** contain at least one element.

Optional Field Presence:

The Protocol Buffer `optional` keyword is used to distinguish between a field being explicitly set versus omitted. This distinction is critical for two scenarios:

1. **Explicit Default Values:** Some fields in the specification define default values that differ from Protocol Buffer's implicit defaults. Implementations should apply the default value when the field is not explicitly provided.

2. **Agent Card Canonicalization:** When creating cryptographic signatures of Agent Cards, it is required to produce a canonical JSON representation. The `optional` keyword enables implementations to distinguish between fields that were explicitly set (and should be included in the canonical form) versus fields that were

omitted (and should be excluded from canonicalization). This ensures Agent Cards can be reconstructed to accurately match their signature.

Unrecognized Fields:

Implementations **SHOULD** ignore unrecognized fields in messages, allowing for forward compatibility as the protocol evolves.

5.8. Custom Binding Identification¶

Custom protocol bindings **SHOULD** be identified by a URI. Using a URI as the identifier provides globally unique identification across all implementers.

The `protocolBinding` field in the Agent Card's `supportedInterfaces` entry **SHOULD** be a URI:

```
{
  "supportedInterfaces": [
    {
      "url": "wss://agent.example.com/a2a/websocket",
      "protocolBinding": "https://example.com/bindings/websocket/v1",
      "protocolVersion": "1.0"
    }
  ]
}
```

When a breaking change is introduced to a binding, a new URI **MUST** be used so that clients can distinguish between incompatible versions:

```
https://example.com/bindings/websocket/v1  →  https://example.com/bindings/websocket/v2
```

6. Common Workflows & Examples¶

This section provides illustrative examples of common A2A interactions across different bindings.

6.1. Basic Task Execution¶

Scenario: Client asks a question and receives a completed task response.

Request:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token

{
  "message": {
    "role": "ROLE_USER",
    "parts": [{"text": "What is the weather today?"}],
    "messageId": "msg-uuid"
  }
}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "task": {
    "id": "task-uuid",
    "contextId": "context-uuid",
    "status": {"state": "TASK_STATE_COMPLETED"},
    "artifacts": [{
      "artifactId": "artifact-uuid",
      "name": "Weather Report",
      "parts": [{"text": "Today will be sunny with a high of 75°F"}]
    }]
  }
}
```

6.2. Streaming Task Execution¶

Scenario: Client requests a long-running task with real-time updates.

Request:

```
POST /message:stream HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token

{
  "message": {
    "role": "ROLE_USER",
    "parts": [{"text": "Write a detailed report on climate change"}],
    "messageId": "msg-uuid"
  }
}
```

SSE Response Stream:

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
```

```
data: {"task": {"id": "task-uuid", "status": {"state": "TASK_STATE_WORKING"}}}

data: {"artifactUpdate": {"taskId": "task-uuid", "artifact": {"parts": [{"text": "# Climate Change Report\n\n"}]}}}

data: {"statusUpdate": {"taskId": "task-uuid", "status": {"state": "TASK_STATE_COMPLETED"}}}
```

6.3. Multi-Turn Interaction¶

Scenario: Agent requires additional input to complete a task.

Initial Request:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token

{
  "message": {
    "role": "ROLE_USER",
    "parts": [{"text": "Book me a flight"}],
    "messageId": "msg-1"
  }
}
```

Response (Input Required):

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "task": {
    "id": "task-uuid",
    "status": {
      "state": "TASK_STATE_INPUT_REQUIRED",
      "message": {
        "role": "ROLE_AGENT",
        "parts": [{"text": "I need more details. Where would you like to fly from and to?"}]
      }
    }
  }
}
```

Follow-up Request:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token

{
  "message": {
    "taskId": "task-uuid",
    "role": "ROLE_USER",
    "parts": [{"text": "From San Francisco to New York"}],
    "messageId": "msg-2"
  }
}
```

6.4. Version Negotiation Error¶

Scenario: Client requests an unsupported protocol version.

Request:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token
A2A-Version: 0.5

{
  "message": {
    "role": "ROLE_USER",
    "parts": [{"text": "Hello"}],
    "messageId": "msg-uuid"
  }
}
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://a2a-protocol.org/errors/version-not-supported",
  "title": "Protocol Version Not Supported",
  "status": 400,
  "detail": "The requested A2A protocol version 0.5 is not supported by this agent",
  "supportedVersions": ["0.3"]
}
```

6.5. Task Listing and Management¶

Scenario: Client wants to see all tasks from a specific context or all tasks with a particular status.

Request: All tasks from a specific context¶

Request:

```
GET /tasks?contextId=c295ea44-7543-4f78-b524-7a38915ad6e4&pageSize=10&historyLength=3 HTTP/1.1
Host: agent.example.com
Authorization: Bearer token
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "tasks": [
    {
      "id": "3f36680c-7f37-4a5f-945e-d78981fafd36",
      "contextId": "c295ea44-7543-4f78-b524-7a38915ad6e4",
      "status": {
        "state": "TASK_STATE_COMPLETED",
        "timestamp": "2024-03-15T10:15:00Z"
      }
    }
  ],
  "totalSize": 5,
  "pageSize": 10,
  "nextPageToken": ""
}
```

Request: All working tasks across all contexts¶

Request:

```
GET /tasks?status=TASK_STATE_WORKING&pageSize=20 HTTP/1.1
Host: agent.example.com
Authorization: Bearer token
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "tasks": [
    {
      "id": "789abc-def0-1234-5678-9abcdef01234",
      "contextId": "another-context-id",
      "status": {
        "state": "TASK_STATE_WORKING",
        "message": {
          "role": "ROLE_AGENT",
          "parts": [
            {
              "text": "Processing your document analysis..."
            }
          ],
          "messageId": "msg-status-update"
        },
        "timestamp": "2024-03-15T10:20:00Z"
      }
    }
  ],
  "totalSize": 1,
  "pageSize": 20,
  "nextPageToken": ""
}
```

Pagination Example¶

Request:

```
GET /tasks?contextId=c295ea44-7543-4f78-b524-7a38915ad6e4&pageSize=10&pageToken=base64-encoded-cursor-token HTTP/1.1
Host: agent.example.com
Authorization: Bearer token
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "tasks": [
    /* ... additional tasks */
  ],
  "totalSize": 15,
  "pageSize": 10,
  "nextPageToken": "base64-encoded-next-cursor-token"
}
```

Validation Error Example¶

Request:

```
GET /tasks?pageSize=150&historyLength=-5&status=TASK_STATE_RUNNING HTTP/1.1
Host: agent.example.com
Authorization: Bearer token
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "status": 400,
  "detail": "Invalid parameters",
  "errors": [
    {
      "field": "pageSize",
      "message": "Must be between 1 and 100 inclusive, got 150"
    },
    {
      "field": "historyLength",
      "message": "Must be non-negative integer, got -5"
    },
    {
      "field": "status",
      "message": "Invalid status value 'TASK_STATE_RUNNING'. Must be one of: TASK_STATE_SUBMITTED, TASK_STATE_WORKING, TASK_STATE_COMPLETED, TASK_STATE_FAI
    }
  ]
}
```

6.6. Push Notification Setup and Usage¶

Scenario: Client requests a long-running report generation and wants to be notified via webhook when it's done.

Initial Request with Push Notification Config:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token

{
  "message": {
    "role": "ROLE_USER",
    "parts": [
      {
        "text": "Generate the Q1 sales report. This usually takes a while. Notify me when it's ready."
      }
    ],
    "messageId": "6dbc13b5-bd57-4c2b-b503-24e381b6c8d6"
  },
  "configuration": {
    "taskPushNotificationConfig": {
      "url": "https://client.example.com/webhook/a2a-notifications",
      "authentication": {
        "scheme": "Bearer",
        "credentials": "secure-client-token-for-task-aaa"
      }
    }
  }
}
```

Response (Task Submitted):

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "task": {
    "id": "43667960-d455-4453-b0cf-1bae4955270d",
    "contextId": "c295ea44-7543-4f78-b524-7a38915ad6e4",
    "status": {
      "state": "TASK_STATE_SUBMITTED",
      "timestamp": "2024-03-15T11:00:00Z"
    }
  }
}
```

Later: Server POSTs Notification to Webhook:

```
POST /webhook/a2a-notifications HTTP/1.1
Host: client.example.com
Authorization: Bearer secure-client-token-for-task-aaa
Content-Type: application/a2a+json

{
  "statusUpdate": {
    "taskId": "43667960-d455-4453-b0cf-1bae4955270d",
    "contextId": "c295ea44-7543-4f78-b524-7a38915ad6e4",
    "status": {
      "state": "TASK_STATE_COMPLETED",
      "timestamp": "2024-03-15T18:30:00Z"
```

```
    }
  }
}
```

6.7. File Exchange (Upload and Download)¶

Scenario: Client sends an image for analysis, and the agent returns a modified image.

Request with File Upload:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token

{
  "message": {
    "role": "ROLE_USER",
    "parts": [
      {
        "text": "Analyze this image and highlight any faces."
      },
      {
        "raw": "iVBORw0KGgoAAAANSUhEUgAAAAUA..."
        "filename": "input_image.png",
        "mediaType": "image/png",
      }
    ],
    "messageId": "6dbc13b5-bd57-4c2b-b503-24e381b6c8d6"
  }
}
```

Response with File Reference:

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "task": {
    "id": "43667960-d455-4453-b0cf-1bae4955270d",
    "contextId": "c295ea44-7543-4f78-b524-7a38915ad6e4",
    "status": {
      "state": "TASK_STATE_COMPLETED",
      "timestamp": "2024-03-15T12:05:00Z"
    },
    "artifacts": [
      {
        "artifactId": "9b6934dd-37e3-4eb1-8766-962efaab63a1",
        "name": "processed_image_with_faces.png",
        "parts": [
          {
            "url": "https://storage.example.com/processed/task-bbb/output.png?token=xyz",
            "filename": "output.png",
            "mediaType": "image/png"
          }
        ]
      }
    ]
  }
}
```

6.8. Structured Data Exchange¶

Scenario: Client asks for a list of open support tickets in a specific JSON format.

Request:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token

{
  "message": {
    "role": "ROLE_USER",
    "parts": [
      {
        "text": "Show me a list of my open IT tickets",
        "metadata": {
          "mediaType": "application/json",
          "schema": {
            "type": "array",
            "items": {
              "type": "object",
              "properties": {
                "ticketNumber": { "type": "string" },
                "description": { "type": "string" }
              }
            }
          }
        }
      }
    ],
    "messageId": "85b26db5-ffbb-4278-a5da-a7b09dea1b47"
```

```
  }
}
```

Response with Structured Data:

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "task": {
    "id": "d8c6243f-5f7a-4f6f-821d-957ce51e856c",
    "contextId": "c295ea44-7543-4f78-b524-7a38915ad6e4",
    "status": {
      "state": "TASK_STATE_COMPLETED",
      "timestamp": "2025-04-17T17:47:09.680794Z"
    },
    "artifacts": [
      {
        "artifactId": "c5e0382f-b57f-4da7-87d8-b85171fad17c",
        "parts": [
          {
            "text": "[{\"ticketNumber\":\"REQ12312\",\"description\":\"request for VPN access\"},{\"ticketNumber\":\"REQ23422\",\"description\":\"Add to DL
          }
        ]
      }
    ]
  }
}
```

6.9. Fetching Authenticated Extended Agent Card¶

Scenario: A client discovers a public Agent Card indicating support for an authenticated extended card and wants to retrieve the full details.

Step 1: Client fetches the public Agent Card:

```
GET /.well-known/agent-card.json HTTP/1.1
Host: example.com
```

Response includes:

```
{
  "capabilities": {
    "extendedAgentCard": true
  },
  "securitySchemes": {
    "google": {
      "openIdConnectSecurityScheme": {
        "openIdConnectUrl": "https://accounts.google.com/.well-known/openid-configuration"
      }
    }
  }
}
```

Step 2: Client obtains credentials (out-of-band OAuth 2.0 flow)¶

Step 3: Client fetches authenticated extended Agent Card¶

```
GET /extendedAgentCard HTTP/1.1
Host: agent.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "name": "Extended Agent with Additional Skills",
  "skills": [
    /* Extended skills available to authenticated users */
  ]
}
```

7. Authentication and Authorization¶

A2A treats agents as standard enterprise applications, relying on established web security practices. Identity information is handled at the protocol layer, not within A2A semantics.

For a comprehensive guide on enterprise security aspects, see Enterprise-Ready Features.

7.1. Protocol Security¶

Production deployments **MUST** use encrypted communication (HTTPS for HTTP-based bindings, TLS for gRPC). Implementations **SHOULD** use modern TLS configurations (TLS 1.3+ recommended) with strong cipher suites.

7.2. Server Identity Verification¶

A2A Clients **SHOULD** verify the A2A Server's identity by validating its TLS certificate against trusted certificate authorities (CAs) during the TLS handshake.

7.3. Client Authentication Process¶

1. **Discovery of Requirements:** The client discovers the server's required authentication schemes via the `securitySchemes` field in the AgentCard.
2. **Credential Acquisition (Out-of-Band):** The client obtains the necessary credentials through an out-of-band process specific to the required authentication scheme.
3. **Credential Transmission:** The client includes these credentials in protocol-appropriate headers or metadata for every A2A request.

7.4. Server Authentication Responsibilities¶

The A2A Server:

- **MUST** authenticate every incoming request based on the provided credentials and its declared authentication requirements.
- **SHOULD** use appropriate binding-specific error codes for authentication challenges or rejections.
- **SHOULD** provide relevant authentication challenge information with error responses.

7.5. Server Authorization Responsibilities¶

Once authenticated, the A2A Server authorizes requests based on the authenticated identity and its own policies. Authorization logic is implementation-specific and **MAY** consider:

- Specific skills requested
- Actions attempted within tasks
- Data access policies
- OAuth scopes (if applicable)

7.6. In-Task Authorization¶

In the course of performing a task, an agent may require authorization to perform some action. Examples include:

- An agent requiring an OAuth access token to call an API or another agent
- An agent requiring human approval before a destructive action is taken

In the sections below, we refer to the object that represents the approved authorization as a credential.

Agents may have multiple means for retrieving this authorization, such as via directly sending messages to a human.

A2A provides the capability for agents to delegate the fulfillment of this authorization to the client via the `TASK_STATE_AUTH_REQUIRED` Task state. This provides agents a fallback path for requesting authorization by passing the responsibility to the client.

7.6.1 In-Task Authorization Agent Responsibilities¶

To request that a client fulfills an authorization request, the agent:

1. MUST use a Task to track the operation it is performing
2. MUST transition the TaskState to `TASK_STATE_AUTH_REQUIRED`
3. MUST include a TaskStatus message explaining the required authorization, unless the details of the authorization have been negotiated out-of-band or via an extension

Agents MUST arrange to receive credentials via an out-of-band means, unless an in-band mechanism has been negotiated out-of-band or via an extension.

If the credential is received out-of-band, the agent SHOULD maintain any active response streams with the client after setting the TaskState to `TASK_STATE_AUTH_REQUIRED`. The agent MAY immediately continue Task processing after receiving the credential, without a requirement that clients send a follow-up message.

Agents SHOULD support receiving messages directed to the Task while the Task remains in `TASK_STATE_AUTH_REQUIRED`. This enables clients to negotiate, correct, or reject an authorization request.

7.6.2 In-Task Authorization Client Responsibilities¶

Upon receiving a Task in `TASK_STATE_AUTH_REQUIRED`, a client is expected to take action in some way to resolve the agent's request for authorization.

A client may:

- Send a response message to the Task to negotiate, correct, or reject the authorization request.
- Contact another human, agent, or service to fulfill the authorization request
- Directly fulfill the authorization request via an out-of-band or extension negotiated means

If the client is itself an A2A agent actively processing a Task, the client may further delegate the authorization request to its client by transitioning its own Task to `TASK_STATE_AUTH_REQUIRED`. The client SHOULD follow all In-Task Authorization Agent Responsibilities. This enables forming a chain of Tasks in `TASK_STATE_AUTH_REQUIRED`.

Clients may not be aware of when the agent receives credentials out-of-band and subsequently continues Task processing. If a client does not have an active response stream open with the agent, the client risks missing Task updates. To avoid this, a client SHOULD perform one of the following:

- Subscribe to a stream of events for the Task using the Subscribe to Task operation
- Register a webhook to receive events, if supported by the agent, using the Create Push Notification Config operation
- Begin polling the Task using the Get Task operation

7.6.3 In-Task Authorization Security Considerations¶

Agents SHOULD receive credentials for in-task authorization requests out of band via a secure channel, such as HTTPS. This ensures that credentials are provided directly to the agent.

In-band credential exchange may be negotiated via out-of-band means or by using extensions. In-band credential exchange can allow credentials to be passed across chains of multiple A2A agents, exposing those credentials to each agent participating in the chain.

If using in-band credential exchange, we recommend adhering to the following security practices:

- Credentials SHOULD be bound to the agent which originated the request, such that only this agent is able to use the credentials. This ensures that credentials propagating through a chain of A2A requests are only usable by the requesting agent.
- Credentials containing sensitive information SHOULD be only readable by the agent which originated the request, such as by encrypting the credential.

8. Agent Discovery: The Agent Card¶

8.1. Purpose¶

A2A Servers **MUST** make an Agent Card available. The Agent Card describes the server's identity, capabilities, skills, and interaction requirements. Clients use this information for discovering suitable agents and configuring interactions.

For more on discovery strategies, see the Agent Discovery guide.

8.2. Discovery Mechanisms¶

Clients can find Agent Cards through:

- **Well-Known URI:** Accessing `https://{server_domain}/.well-known/agent-card.json` (see Section 8.6 for caching guidance)
- **Registries/Catalogs:** Querying curated catalogs of agents
- **Direct Configuration:** Pre-configured Agent Card URLs or content

8.3. Protocol Declaration Requirements¶

The AgentCard **MUST** properly declare supported protocols:

8.3.1. Supported Interfaces Declaration¶

- The `supportedInterfaces` field **SHOULD** declare all supported protocol combinations in preference order
- The first entry in `supportedInterfaces` represents the preferred interface
- Each interface **MUST** accurately declare its transport protocol and URL
- URLs **MAY** be reused if multiple transports are available at the same endpoint

8.3.2. Client Protocol Selection¶

Clients **MUST** follow these rules:

1. Parse `supportedInterfaces` if present, and select the first supported transport
2. Prefer earlier entries in the ordered list when multiple options are supported
3. Use the correct URL for the selected transport
4. Set the `tenant` field in every request message to exactly the value declared in the selected `AgentInterface` entry (omit the field if `tenant` is not set in that entry)

8.4. Agent Card Signing¶

Agent Cards **MAY** be digitally signed using JSON Web Signature (JWS) as defined in RFC 7515 to ensure authenticity and integrity. Signatures allow clients to verify that an Agent Card has not been tampered with and originates from the claimed provider.

8.4.1. Canonicalization Requirements¶

Before signing, the Agent Card content **MUST** be canonicalized using the JSON Canonicalization Scheme (JCS) as defined in RFC 8785. This ensures consistent signature generation and verification across different JSON implementations.

Canonicalization Rules:

1. **Field Presence and Default Value Handling**: Before canonicalization, the JSON representation **MUST** respect Protocol Buffer field presence semantics as defined in Section 5.7. This ensures that the canonical form accurately reflects which fields were explicitly provided versus which were omitted, enabling signature verification when Agent Cards are reconstructed:
    - **Optional fields not explicitly set**: Fields marked with the `optional` keyword that were not explicitly set **MUST** be omitted from the JSON object
    - **Optional fields explicitly set to defaults**: Fields marked with `optional` that were explicitly set to a value (even if that value matches a default) **MUST** be included in the JSON object
    - **Required fields**: Fields marked with `REQUIRED` **MUST** always be present, even if the field value matches the default.
    - **Default values**: Fields with default values **MUST** be omitted unless the field is marked as `REQUIRED` or has the `optional` keyword.
2. **RFC 8785 Compliance**: The Agent Card JSON **MUST** be canonicalized according to RFC 8785, which specifies:
    - Predictable ordering of object properties (lexicographic by key)
    - Consistent representation of numbers, strings, and other primitive values
    - Removal of insignificant whitespace
3. **Signature Field Exclusion**: The `signatures` field itself **MUST** be excluded from the content being signed to avoid circular dependencies.

Example of Default Value Removal:

Original Agent Card fragment:

```
{
  "name": "Example Agent",
  "description": "",
  "capabilities": {
    "streaming": false,
    "pushNotifications": false,
    "extensions": []
  },
  "skills": []
}
```

Applying the canonicalization rules:

- `name`: "Example Agent" - REQUIRED field → **include**
- `description`: "" - REQUIRED field → **include**
- `capabilities`: object - REQUIRED field → **include** (after processing children)
  - `streaming`: false - optional field, present in JSON (explicitly set) → **include**
  - `pushNotifications`: false - optional field, present in JSON (explicitly set) → **include**
  - `extensions`: [] - repeated field (not REQUIRED) with empty array → **omit**
- `skills`: [] - REQUIRED field → **include**

After applying RFC 8785:

```
{"capabilities":{"pushNotifications":false,"streaming":false},"description":"","name":"Example Agent","skills":[]}
```

8.4.2. Signature Format¶

Signatures use the JSON Web Signature (JWS) format as defined in RFC 7515. The `AgentCardSignature` object represents JWS components using three fields:

- **`protected`** (required, string): Base64url-encoded JSON object containing the JWS Protected Header
- **`signature`** (required, string): Base64url-encoded signature value
- **`header`** (optional, object): JWS Unprotected Header as a JSON object (not base64url-encoded)

JWS Protected Header Parameters:

The protected header **MUST** include:

- `alg`: Algorithm used for signing (e.g., "ES256", "RS256")
- `typ`: **SHOULD** be set to "JOSE" for JWS
- `kid`: Key ID for identifying the signing key

The protected header **MAY** include:

- `jku`: URL to JSON Web Key Set (JWKS) containing the public key

Signature Generation Process:

1. **Prepare the payload:**
   - Remove properties with default values from the Agent Card
   - Exclude the `signatures` field
   - Canonicalize the resulting JSON using RFC 8785 to produce the canonical payload
2. **Create the protected header:**
   - Construct a JSON object with the required header parameters (`alg`, `typ`, `kid`) and any optional parameters (`jku`)
   - Serialize the header to JSON
   - Base64url-encode the serialized header to produce the `protected` field value
3. **Compute the signature:**
   - Construct the JWS Signing Input: `ASCII(BASE64URL(UTF8(JWS Protected Header)) || '.' || BASE64URL(JWS Payload))`
   - Sign the JWS Signing Input using the algorithm specified in the `alg` header parameter and the private key
   - Base64url-encode the resulting signature bytes to produce the `signature` field value
4. **Assemble the AgentCardSignature:**
   - Set `protected` to the base64url-encoded protected header from step 2
   - Set `signature` to the base64url-encoded signature value from step 3
   - Optionally set `header` to a JSON object containing any unprotected header parameters.

Example:

Given a canonical Agent Card payload and signing key, the signature generation produces:

```
{
  "protected": "eyJhbGciOiJFUzI1NiIsInR5cCI6IkpPU0UiLCJraWQiOiJrZXktMSIsImprdSI6Imh0dHBzOi8vZXhhbXBsZS5jb20vYWdlbnQvandrcy5qc29uIn0",
  "signature": "QFdkNLNszlGj3z3u0YQGt_T9LixY3qtdQpZmsTdDHDe3fXV9y9-B3m2-XgCpzuhiLt8E0tV6HXoZKHv4GtHgKQ"
}
```

Where the `protected` value decodes to:

```
{"alg":"ES256","typ":"JOSE","kid":"key-1","jku":"https://example.com/agent/jwks.json"}
```

8.4.3. Signature Verification¶

Clients verifying Agent Card signatures **MUST**:

1. Extract the signature from the `signatures` array
2. Retrieve the public key using the `kid` and `jku` (or from a trusted key store)
3. Remove properties with default values from the received Agent Card
4. Exclude the `signatures` field
5. Canonicalize the resulting JSON using RFC 8785
6. Verify the signature against the canonicalized payload

Security Considerations:

- Clients **SHOULD** verify at least one signature before trusting an Agent Card
- Public keys **SHOULD** be retrieved over secure channels (HTTPS)
- Clients **MAY** maintain a trusted key store for known agent providers
- Expired or revoked keys **MUST NOT** be used for verification
- Multiple signatures **MAY** be present to support key rotation

8.5. Sample Agent Card¶


```
{
  "name": "GeoSpatial Route Planner Agent",
  "description": "Provides advanced route planning, traffic analysis, and custom map generation services. This agent can calculate optimal routes, estimate
  "supportedInterfaces": [
    {"url": "https://georoute-agent.example.com/a2a/v1", "protocolBinding": "JSONRPC", "protocolVersion": "1.0"},
    {"url": "https://georoute-agent.example.com/a2a/grpc", "protocolBinding": "GRPC", "protocolVersion": "1.0"},
    {"url": "https://georoute-agent.example.com/a2a/json", "protocolBinding": "HTTP+JSON", "protocolVersion": "1.0"}
  ],
  "provider": {
    "organization": "Example Geo Services Inc.",
    "url": "https://www.examplegeoservices.com"
  },
  "iconUrl": "https://georoute-agent.example.com/icon.png",
  "version": "1.2.0",
  "documentationUrl": "https://docs.examplegeoservices.com/georoute-agent/api",
  "capabilities": {
    "streaming": true,
    "pushNotifications": true,
    "extendedAgentCard": true
  },
  "securitySchemes": {
    "google": {
      "openIdConnectSecurityScheme": {
        "openIdConnectUrl": "https://accounts.google.com/.well-known/openid-configuration"
      }
    }
  },
  "security": [{ "google": ["openid", "profile", "email"] }],
  "defaultInputModes": ["application/json", "text/plain"],
  "defaultOutputModes": ["application/json", "image/png"],
  "skills": [
    {
      "id": "route-optimizer-traffic",
      "name": "Traffic-Aware Route Optimizer",
      "description": "Calculates the optimal driving route between two or more locations, taking into account real-time traffic conditions, road closures,
      "tags": ["maps", "routing", "navigation", "directions", "traffic"],
      "examples": [
        "Plan a route from '1600 Amphitheatre Parkway, Mountain View, CA' to 'San Francisco International Airport' avoiding tolls.",
        "{\"origin\": {\"lat\": 37.422, \"lng\": -122.084}, \"destination\": {\"lat\": 37.7749, \"lng\": -122.4194}, \"preferences\": [\"avoid_ferries\"]}"
      ],
      "inputModes": ["application/json", "text/plain"],
      "outputModes": [
        "application/json",
        "application/vnd.geo+json",
        "text/html"
      ]
    },
    {
      "id": "custom-map-generator",
      "name": "Personalized Map Generator",
      "description": "Creates custom map images or interactive map views based on user-defined points of interest, routes, and style preferences. Can overl
      "tags": ["maps", "customization", "visualization", "cartography"],
      "examples": [
        "Generate a map of my upcoming road trip with all planned stops highlighted.",
        "Show me a map visualizing all coffee shops within a 1-mile radius of my current location."
      ],
      "inputModes": ["application/json"],
      "outputModes": [
        "image/png",
        "image/jpeg",
        "application/json",
        "text/html"
      ]
    }
  ],
  "signatures": [
    {
      "protected": "eyJhbGciOiJFUzI1NiIsInR5cCI6IkpPU0UiLCJraWQiOiJrZXktMSIsImprdSI6Imh0dHBzOi8vZXhhbXBsZS5jb20vYWdlbnQvandrcy5qc29uIn0",
      "signature": "QFdkNLNszlGj3z3u0YQGt_T9LixY3qtdQpZmsTdDHDe3fXV9y9-B3m2-XgCpzuhiLt8E0tV6HXoZKHv4GtHgKQ"
    }
  ]
}
```


8.6. Caching¶

Agent Card content changes infrequently relative to the frequency at which clients may fetch it. Servers and clients **SHOULD** use standard HTTP caching mechanisms to reduce unnecessary network overhead.

8.6.1. Server Requirements¶

- Agent Card HTTP endpoints **SHOULD** include a `Cache-Control` response header with a `max-age` directive appropriate for the agent's expected update frequency
- Agent Card HTTP endpoints **SHOULD** include an `ETag` response header derived from the Agent Card's `version` field or a hash of the card content
- Agent Card HTTP endpoints **MAY** include a `Last-Modified` response header

8.6.2. Client Requirements¶

- Clients **SHOULD** honor HTTP caching semantics as defined in RFC 9111 when fetching Agent Cards
- When a cached Agent Card has expired, clients **SHOULD** use conditional requests (`If-None-Match` with the stored `ETag`, or `If-Modified-Since`) to avoid re-downloading unchanged cards
- When the server does not include caching headers, clients **MAY** apply an implementation-specific default cache duration

9. JSON-RPC Protocol Binding¶

The JSON-RPC protocol binding provides a simple, HTTP-based interface using JSON-RPC 2.0 for method calls and Server-Sent Events for streaming.

9.1. Protocol Requirements¶

- **Protocol:** JSON-RPC 2.0 over HTTP(S)
- **Content-Type:** `application/json` for requests and responses
- **Method Naming:** PascalCase method names matching gRPC conventions (e.g., `SendMessage`, `GetTask`)
- **Streaming:** Server-Sent Events (`text/event-stream`)

9.2. Service Parameter Transmission¶

A2A service parameters defined in Section 3.2.6 **MUST** be transmitted using standard HTTP request headers, as JSON-RPC 2.0 operates over HTTP(S).

Service Parameter Requirements:

- Service parameter names **MUST** be transmitted as HTTP header fields
- Service parameter keys are case-insensitive per HTTP specification (RFC 7230)
- Multiple values for the same service parameter (e.g., `A2A-Extensions`) **SHOULD** be comma-separated in a single header field

Example Request with A2A Service Parameters:

```
POST /rpc HTTP/1.1
Host: agent.example.com
Content-Type: application/json
Authorization: Bearer token
A2A-Version: 0.3
A2A-Extensions: https://example.com/extensions/geolocation/v1,https://standards.org/extensions/citations/v1

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "SendMessage",
  "params": { /* SendMessageRequest */ }
}
```

9.3. Base Request Structure¶

All JSON-RPC requests **MUST** follow the standard JSON-RPC 2.0 format:

```
{
  "jsonrpc": "2.0",
  "id": "unique-request-id",
  "method": "category/action",
  "params": { /* method-specific parameters */ }
}
```

9.4. Core Methods¶

9.4.1. `SendMessage`¶

Sends a message to initiate or continue a task.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "SendMessage",
  "params": { /* SendMessageRequest object */ }
}
```

Referenced Objects: `SendMessageRequest`, `Message`

Response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    /* SendMessageResponse object, contains one of:
     * "task": { Task object }
     * "message": { Message object }
    */
  }
```

Referenced Objects: `Task`, `Message`

9.4.2. `SendStreamingMessage`¶

Sends a message and subscribes to real-time updates via Server-Sent Events.

Request: Same as `SendMessage`

Response: HTTP 200 with `Content-Type: text/event-stream`

```
data: {"jsonrpc": "2.0", "id": 1, "result": { /* StreamResponse object */ }}

data: {"jsonrpc": "2.0", "id": 1, "result": { /* StreamResponse object */ }}
```

Referenced Objects: `StreamResponse`

9.4.3. `GetTask`¶

Retrieves the current state of a task.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "GetTask",
  "params": {
    "id": "task-uuid",
    "historyLength": 10
  }
}
```

9.4.4. `ListTasks`¶

Lists tasks with optional filtering and pagination.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "ListTasks",
  "params": {
    "contextId": "context-uuid",
    "status": "TASK_STATE_WORKING",
    "pageSize": 50,
    "pageToken": "cursor-token"
  }
}
```

9.4.5. `CancelTask`¶

Cancels an ongoing task.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "method": "CancelTask",
  "params": {
    "id": "task-uuid"
  }
}
```

9.4.6. `SubscribeToTask`¶

Subscribes to a task stream for receiving updates on a task that is not in a terminal state.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "method": "SubscribeToTask",
  "params": {
    "id": "task-uuid"
  }
}
```

Response: SSE stream (same format as `SendStreamingMessage`)

Error: Returns `UnsupportedOperationError` if the task is in a terminal state (`TASK_STATE_COMPLETED`, `TASK_STATE_FAILED`, `TASK_STATE_CANCELED`, or `TASK_STATE_REJECTED`).

9.4.7. Push Notification Configuration Methods¶

- `CreateTaskPushNotificationConfig` - Create push notification configuration
- `GetTaskPushNotificationConfig` - Get push notification configuration
- `ListTaskPushNotificationConfigs` - List push notification configurations
- `DeleteTaskPushNotificationConfig` - Delete push notification configuration

9.4.8. `GetExtendedAgentCard`¶

Retrieves an extended Agent Card.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 6,
  "method": "GetExtendedAgentCard"
}
```

9.5. Error Handling¶

JSON-RPC error responses use the standard JSON-RPC 2.0 error object structure, which maps to the generic A2A error model defined in Section 3.3.2 as follows:

- **Error Code**: Mapped to `error.code` (numeric JSON-RPC error code)
- **Error Message**: Mapped to `error.message` (human-readable string)
- **Error Details**: Mapped to `error.data` (array of objects, each containing a `@type` key, using ProtoJSON `Any` representation)

Standard JSON-RPC Error Codes:

JSON-RPC Error Code	Error Name	Standard Message	Description
`-32700`	`JSONParseError`	"Invalid JSON payload"	The server received invalid JSON
`-32600`	`InvalidRequestError`	"Request payload validation error"	The JSON sent is not a valid Request object
`-32601`	`MethodNotFoundError`	"Method not found"	The requested method does not exist or is not available
`-32602`	`InvalidParamsError`	"Invalid parameters"	The method parameters are invalid
`-32603`	`InternalError`	"Internal error"	An internal error occurred on the server

A2A-Specific Error Codes:

A2A-specific errors use codes in the range `-32001` to `-32099`. For the complete mapping of A2A error types to JSON-RPC error codes, see Section 5.4 (Error Code Mappings).

Error Detail Objects:

Each object in the `data` array **MUST** include a `@type` key that identifies the object's type. Implementations **SHOULD** use well-known types such as `google.rpc.ErrorInfo` to refine error reporting, or `google.rpc.BadRequest` to attach structured data to validation errors. Additional error context **MAY** be included as further objects in the `data` array.

Error Response Structure:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid parameters",
    "data": [
      {
        "@type": "type.googleapis.com/google.rpc.BadRequest",
        "fieldViolations": [
          {
            "field": "message.parts",
            "description": "At least one part is required"
          }
        ]
      }
    ]
  }
}
```

Example A2A-Specific Error Response:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "error": {
    "code": -32001,
    "message": "Task not found",
    "data": [
      {
        "@type": "type.googleapis.com/google.rpc.ErrorInfo",
        "reason": "TASK_NOT_FOUND",
        "domain": "a2a-protocol.org",
```

```
        "metadata": {
          "taskId": "nonexistent-task-id",
          "timestamp": "2025-11-09T10:30:00.000Z"
        }
      }
    ]
  }
}
```

10. gRPC Protocol Binding¶

The gRPC Protocol Binding provides a high-performance, strongly-typed interface using Protocol Buffers over HTTP/2. The gRPC Protocol Binding leverages the API guidelines to simplify gRPC to HTTP mapping.

10.1. Protocol Requirements¶

- **Protocol:** gRPC over HTTP/2 with TLS
- **Definition:** Use the normative Protocol Buffers definition in `specification/a2a.proto`
- **Serialization:** Protocol Buffers version 3
- **Service:** Implement the `A2AService` gRPC service

10.2. Service Parameter Transmission¶

A2A service parameters defined in Section 3.2.6 **MUST** be transmitted using gRPC metadata (headers).

Service Parameter Requirements:

- Service parameter names **MUST** be transmitted as gRPC metadata keys
- Metadata keys are case-insensitive and automatically converted to lowercase by gRPC
- Multiple values for the same service parameter (e.g., `A2A-Extensions`) **SHOULD** be comma-separated in a single metadata entry

Example gRPC Request with A2A Service Parameters:

```
// Go example using gRPC metadata
md := metadata.Pairs(
    "authorization", "Bearer token",
    "a2a-version", "0.3",
    "a2a-extensions", "https://example.com/extensions/geolocation/v1,https://standards.org/extensions/citations/v1",
)
ctx := metadata.NewOutgoingContext(context.Background(), md)

// Make the RPC call with the context containing metadata
response, err := client.SendMessage(ctx, request)
```

Metadata Handling:

- Implementations **MUST** extract A2A service parameters from gRPC metadata for processing
- Servers **SHOULD** validate required service parameters (e.g., `A2A-Version`) from metadata
- Service parameter keys in metadata are normalized to lowercase per gRPC conventions

10.3. Service Definition¶

Method	Request	Response	Description
`SendMessage`	`SendMessageRequest`	`SendMessageResponse`	Sends a message to an agent.
`SendStreamingMessage`	`SendMessageRequest`	stream `StreamResponse`	Sends a streaming message to an agent, allowing for real-time interaction and status updates. Streaming version of `SendMessage`
`GetTask`	`GetTaskRequest`	`Task`	Gets the latest state of a task.
`ListTasks`	`ListTasksRequest`	`ListTasksResponse`	Lists tasks that match the specified filter.
`CancelTask`	`CancelTaskRequest`	`Task`	Cancels a task in progress.
`SubscribeToTask`	`SubscribeToTaskRequest`	stream `StreamResponse`	Subscribes to task updates for tasks not in a terminal state. Returns `UnsupportedOperationError` if the task is already in a terminal state (completed, failed, canceled, rejected).
`CreateTaskPushNotificationConfig`	`TaskPushNotificationConfig`	`TaskPushNotificationConfig`	Creates a push notification config for a task.
`GetTaskPushNotificationConfig`	`GetTaskPushNotificationConfigRequest`	`TaskPushNotificationConfig`	Gets a push notification config for a task.
`ListTaskPushNotificationConfigs`	`ListTaskPushNotificationConfigsRequest`	`ListTaskPushNotificationConfigsResponse`	Get a list of push notifications configured for a task.
`GetExtendedAgentCard`	`GetExtendedAgentCardRequest`	`AgentCard`	Gets the extended agent card for the authenticated agent.
`DeleteTaskPushNotificationConfig`	`DeleteTaskPushNotificationConfigRequest`	`empty`	Deletes a push notification config for a task.

10.4. Core Methods¶

10.4.1. SendMessage¶

Sends a message to an agent.

Request:

Represents a request for the `SendMessage` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`message`	`Message`	Yes	The message to send to the agent.
`configuration`	`SendMessageConfiguration`	No	Configuration for the send request.
`metadata`	`object`	No	A flexible key-value map for passing additional context or parameters.

Response:

Represents the response for the `SendMessage` method.

Field	Type	Required	Description
`task`	`Task`	Optional (OneOf)	The task created or updated by the message.
`message`	`Message`	Optional (OneOf)	A message from the agent.

Note: A `SendMessageResponse` MUST contain exactly one of the following: `task`, `message`

10.4.2. SendStreamingMessage¶

Sends a message with streaming updates.

Request:

Represents a request for the `SendMessage` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`message`	`Message`	Yes	The message to send to the agent.
`configuration`	`SendMessageConfiguration`	No	Configuration for the send request.
`metadata`	`object`	No	A flexible key-value map for passing additional context or parameters.

Response: Server streaming `StreamResponse` objects.

10.4.3. GetTask¶

Retrieves task status.

Request:

Represents a request for the `GetTask` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`id`	`string`	Yes	The resource ID of the task to retrieve.
`historyLength`	`integer`	No	The maximum number of most recent messages from the task's history to retrieve. An unset value means the client does not impose any limit. A value of zero is a request to not include any messages. The server MUST NOT return more messages than the provided value, but MAY apply a lower limit.

Response: See `Task` object definition.

10.4.4. ListTasks¶

Lists tasks with filtering.

Request:

Parameters for listing tasks with optional filtering criteria.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`contextId`	`string`	No	Filter tasks by context ID to get tasks from a specific conversation or session.
`status`	`TaskState`	No	Filter tasks by their current status state.
`pageSize`	`integer`	No	The maximum number of tasks to return. The service may return fewer than this value. If unspecified, at most 50 tasks will be returned. The minimum value is 1. The maximum value is 100.
`pageToken`	`string`	No	A page token, received from a previous `ListTasks` call. `ListTasksResponse.next_page_token`. Provide this to retrieve the subsequent page.
`historyLength`	`integer`	No	The maximum number of messages to include in each task's history.
`statusTimestampAfter`	`timestamp`	No	Filter tasks which have a status updated after the provided timestamp in ISO 8601 format (e.g., "2023-10-27T10:00:00Z"). Only tasks with a status timestamp time greater than or equal to this value will be returned.
`includeArtifacts`	`boolean`	No	Whether to include artifacts in the returned tasks. Defaults to false to reduce payload size.

Response:

Result object for `ListTasks` method containing an array of tasks and pagination information.

Field	Type	Required	Description
`tasks`	array of `Task`	Yes	Array of tasks matching the specified criteria.
`nextPageToken`	`string`	Yes	A token to retrieve the next page of results, or empty if there are no more results in the list.
`pageSize`	`integer`	Yes	The page size used for this response.
`totalSize`	`integer`	Yes	Total number of tasks available (before pagination).

10.4.5. CancelTask¶

Cancels a running task.

Request:

Represents a request for the `CancelTask` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`id`	`string`	Yes	The resource ID of the task to cancel.
`metadata`	`object`	No	A flexible key-value map for passing additional context or parameters.

Response: See `Task` object definition.

10.4.6. SubscribeToTask¶

Subscribe to task updates via streaming. Returns `UnsupportedOperationError` if the task is in a terminal state.

Request:

Represents a request for the `SubscribeToTask` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`id`	`string`	Yes	The resource ID of the task to subscribe to.

Response: Server streaming `StreamResponse` objects.

10.4.7. CreateTaskPushNotificationConfig¶

Creates a push notification configuration for a task.

Request:

Error: Message `CreateTaskPushNotificationConfigRequest` not found.

Response: See `PushNotificationConfig` object definition.

10.4.8. GetTaskPushNotificationConfig¶

Retrieves an existing push notification configuration for a task.

Request:

Represents a request for the `GetTaskPushNotificationConfig` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`taskId`	`string`	Yes	The parent task resource ID.
`id`	`string`	Yes	The resource ID of the configuration to retrieve.

Response: See `PushNotificationConfig` object definition.

10.4.9. ListTaskPushNotificationConfigs¶

Lists all push notification configurations for a task.

Request:

Represents a request for the `ListTaskPushNotificationConfigs` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`taskId`	`string`	Yes	The parent task resource ID.
`pageSize`	`integer`	No	The maximum number of configurations to return.
`pageToken`	`string`	No	A page token received from a previous `ListTaskPushNotificationConfigsRequest` call.

Response:

Represents a successful response for the `ListTaskPushNotificationConfigs` method.

Field	Type	Required	Description
`configs`	array of `TaskPushNotificationConfig`	No	The list of push notification configurations.
`nextPageToken`	`string`	No	A token to retrieve the next page of results, or empty if there are no more results in the list.

10.4.10. DeleteTaskPushNotificationConfig¶

Removes a push notification configuration for a task.

Request:

Represents a request for the `DeleteTaskPushNotificationConfig` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`taskId`	`string`	Yes	The parent task resource ID.
`id`	`string`	Yes	The resource ID of the configuration to delete.

Response: `google.protobuf.Empty`

10.4.11. GetExtendedAgentCard¶

Retrieves the agent's extended capability card after authentication.

Request:

Represents a request for the `GetExtendedAgentCard` method.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.

Response: See `AgentCard` object definition.

10.5. gRPC-Specific Data Types¶

10.5.1. TaskPushNotificationConfig¶

Resource wrapper for push notification configurations. This is a gRPC-specific type used in resource-oriented operations to provide the full resource name along with the configuration data.

A container associating a push notification configuration with a specific task.

Field	Type	Required	Description
`tenant`	`string`	No	Optional. Opaque routing identifier. Must match the `tenant` value from the selected `AgentInterface` in the Agent Card when that field is set.
`id`	`string`	No	The push notification configuration details. A unique identifier (e.g. UUID) for this push notification configuration.
`taskId`	`string`	No	The ID of the task this configuration is associated with.
`url`	`string`	Yes	The URL where the notification should be sent.
`token`	`string`	No	A token unique for this task or session.
`authentication`	`AuthenticationInfo`	No	Authentication information required to send the notification.

10.6. Error Handling¶

gRPC error responses use the standard gRPC status structure with google.rpc.Status, which maps to the generic A2A error model defined in Section 3.3.2 as follows:

- **Error Code**: Mapped to `status.code` (gRPC status code enum)
- **Error Message**: Mapped to `status.message` (human-readable string)
- **Error Details**: Mapped to `status.details` (repeated google.protobuf.Any messages)

A2A Error Representation:

For A2A-specific errors, implementations **MUST** include a `google.rpc.ErrorInfo` message in the `status.details` array with:

- `reason`: The A2A error type in UPPER_SNAKE_CASE without the "Error" suffix (e.g., `TASK_NOT_FOUND`)
- `domain`: Set to `"a2a-protocol.org"`
- `metadata`: Optional map of additional error context

For the complete mapping of A2A error types to gRPC status codes, see Section 5.4 (Error Code Mappings).

Error Response Example:

```
// Standard gRPC invalid argument error
status {
  code: INVALID_ARGUMENT
  message: "Invalid request parameters"
  details: [
```

```
    {
      type: "type.googleapis.com/google.rpc.BadRequest"
      field_violations: [
        {
          field: "message.parts"
          description: "At least one part is required"
        }
      ]
    }
  ]
}
```

Example A2A-Specific Error Response:

```
// A2A-specific task not found error
status {
  code: NOT_FOUND
  message: "Task with ID 'task-123' not found"
  details: [
    {
      type: "type.googleapis.com/google.rpc.ErrorInfo"
      reason: "TASK_NOT_FOUND"
      domain: "a2a-protocol.org"
      metadata: {
        task_id: "task-123"
        timestamp: "2025-11-09T10:30:00Z"
      }
    }
  ]
}
```

10.7. Streaming¶

gRPC streaming uses server streaming RPCs for real-time updates. The `StreamResponse` message provides a union of possible streaming events:

A wrapper object used in streaming operations to encapsulate different types of response data.

Field	Type	Required	Description
`task`	`Task`	Optional (OneOf)	A Task object containing the current state of the task.
`message`	`Message`	Optional (OneOf)	A Message object containing a message from the agent.
`statusUpdate`	`TaskStatusUpdateEvent`	Optional (OneOf)	An event indicating a task status update.
`artifactUpdate`	`TaskArtifactUpdateEvent`	Optional (OneOf)	An event indicating a task artifact update.

Note: A `StreamResponse` MUST contain exactly one of the following: `task`, `message`, `statusUpdate`, `artifactUpdate`

11. HTTP+JSON/REST Protocol Binding¶

The HTTP+JSON protocol binding provides a RESTful interface using standard HTTP methods and JSON payloads.

11.1. Protocol Requirements¶

- **Protocol:** HTTP(S) with JSON payloads
- **Content-Type:** application/a2a+json **SHOULD** be used for requests and responses
- **Methods:** Standard HTTP verbs (GET, POST, PUT, DELETE)
- **URL Patterns:** RESTful resource-based URLs
- **Streaming:** Server-Sent Events for real-time updates

11.2. Service Parameter Transmission¶

A2A service parameters defined in Section 3.2.6 **MUST** be transmitted using standard HTTP request headers.

Service Parameter Requirements:

- Service parameter names **MUST** be transmitted as HTTP header fields
- Service parameter keys are case-insensitive per HTTP specification (RFC 9110)
- Multiple values for the same service parameter (e.g., `A2A-Extensions`) **SHOULD** be comma-separated in a single header field

Example Request with A2A Service Parameters:

```
POST /message:send HTTP/1.1
Host: agent.example.com
Content-Type: application/a2a+json
Authorization: Bearer token
A2A-Version: 0.3
A2A-Extensions: https://example.com/extensions/geolocation/v1,https://standards.org/extensions/citations/v1

{
  "message": {
    "role": "ROLE_USER",
    "parts": [{"text": "Find restaurants near me"}]
  }
}
```

11.3. URL Patterns and HTTP Methods¶

11.3.1. Message Operations¶

- `POST /message:send` - Send message
- `POST /message:stream` - Send message with streaming (SSE response)

11.3.2. Task Operations¶

- `GET /tasks/{id}` - Get task status
- `GET /tasks` - List tasks (with query parameters)
- `POST /tasks/{id}:cancel` - Cancel task
- `POST /tasks/{id}:subscribe` - Subscribe to task updates (SSE response, returns error for terminal tasks)

11.3.3. Push Notification Configuration¶

- `POST /tasks/{id}/pushNotificationConfigs` - Create configuration
- `GET /tasks/{id}/pushNotificationConfigs/{configId}` - Get configuration
- `GET /tasks/{id}/pushNotificationConfigs` - List configurations
- `DELETE /tasks/{id}/pushNotificationConfigs/{configId}` - Delete configuration

11.3.4. Agent Card¶

- `GET /extendedAgentCard` - Get authenticated extended Agent Card

11.4. Request/Response Format¶

All requests and responses use JSON objects structurally equivalent to the Protocol Buffer definitions.

Example Send Message:

```
POST /message:send
Content-Type: application/a2a+json

{
  "message": {
    "messageId": "uuid",
    "role": "ROLE_USER",
    "parts": [{"text": "Hello"}]
  },
  "configuration": {
    "acceptedOutputModes": ["text/plain"]
  }
}
```

Referenced Objects: `SendMessageRequest`, `Message`

Response:

```
HTTP/1.1 200 OK
Content-Type: application/a2a+json

{
  "task": {
    "id": "task-uuid",
    "contextId": "context-uuid",
    "status": {
      "state": "TASK_STATE_COMPLETED"
    }
  }
}
```

Referenced Objects: `Task`

11.5. Query Parameter Naming for Request Parameters¶

HTTP methods that do not support request bodies (GET, DELETE) **MUST** transmit operation request parameters as path parameters or query parameters. This section defines how to map Protocol Buffer field names to query parameter names.

Naming Convention:

Query parameter names **MUST** use `camelCase` to match the JSON serialization of Protocol Buffer field names. This ensures consistency with request bodies used in POST operations.

Example Mappings:

Protocol Buffer Field	Query Parameter Name	Example Usage
`context_id`	`contextId`	`?contextId=uuid`
`page_size`	`pageSize`	`?pageSize=50`
`page_token`	`pageToken`	`?pageToken=cursor`
`task_id`	`taskId`	`?taskId=uuid`

Usage Examples:

List tasks with filtering:

```
GET /tasks?contextId=uuid&status=TASK_STATE_WORKING&pageSize=50&pageToken=cursor
```

Get task with history:

```
GET /tasks/{id}?historyLength=10
```

Field Type Handling:

- **Strings**: Passed directly as query parameter values
- **Booleans**: Represented as lowercase strings (`true`, `false`)
- **Numbers**: Represented as decimal strings
- **Enums**: Represented using their string values (e.g., `status=TASK_STATE_WORKING`)
- **Repeated Fields**: Multiple values **MAY** be passed by repeating the parameter name (e.g., `?tag=value1&tag=value2`) or as comma-separated values (e.g., `?tag=value1,value2`)
- **Nested Objects**: Not supported in query parameters; operations requiring nested objects **MUST** use POST with a request body
- **Datetimes/Timestamps**: Represented as ISO 8601 strings (e.g., `2025-11-09T10:30:00Z`)

URL Encoding:

All query parameter values **MUST** be properly URL-encoded per RFC 3986.

11.6. Error Handling¶

HTTP error responses use the google.rpc.Status JSON representation, which maps to the generic A2A error model defined in Section 3.3.2 as follows:

- **Error Code**: Mapped to the HTTP status code and the `error.code` field
- **Error Message**: Mapped to the `error.message` field (human-readable string)
- **Error Details**: Mapped to the `error.details` array (array of objects, each containing a `@type` key, using ProtoJSON `Any` representation)

Error Detail Objects:

Each object in the `details` array **MUST** include a `@type` key that identifies the object's type. Implementations **SHOULD** use well-known types such as `google.rpc.BadRequest` to attach structured data to validation errors. Additional error context **MAY** be included as further objects in the `details` array.

A2A Error Representation:

Since multiple A2A error types may map to the same HTTP status code (e.g., `TaskNotCancelableError` and `PushNotificationNotSupportedError` both map to `400 Bad Request`), implementations **MUST** include a `google.rpc.ErrorInfo` object in the `details` array for A2A-specific errors with:

- `@type`: Set to `"type.googleapis.com/google.rpc.ErrorInfo"`
- `reason`: The A2A error type in UPPER_SNAKE_CASE without the "Error" suffix (e.g., `TASK_NOT_FOUND`, `TASK_NOT_CANCELABLE`)
- `domain`: Set to `"a2a-protocol.org"`

For the complete mapping of A2A error types to HTTP status codes, see Section 5.4 (Error Code Mappings).

Error Response Example:

```
HTTP/1.1 404 Not Found
Content-Type: application/a2a+json

{
  "error": {
    "code": 404,
    "status": "NOT_FOUND",
    "message": "The specified task ID does not exist or is not accessible",
    "details": [
      {
        "@type": "type.googleapis.com/google.rpc.ErrorInfo",
        "reason": "TASK_NOT_FOUND",
        "domain": "a2a-protocol.org",
        "metadata": {
          "taskId": "task-123",
          "timestamp": "2025-11-09T10:30:00.000Z"
        }
      }
    ]
  }
}
```

Extension fields like `taskId` and `timestamp` provide additional context to help diagnose the error.

11.7. Streaming¶

REST streaming uses Server-Sent Events with the `data` field containing JSON serializations of the protocol data objects:

```
POST /message:stream
Content-Type: application/a2a+json

{ /* SendMessageRequest object */ }
```

Referenced Objects: `SendMessageRequest`

Response:

```
HTTP/1.1 200 OK
Content-Type: text/event-stream

data: { /* StreamResponse object */ }

data: { /* StreamResponse object */ }
```

Referenced Objects: `StreamResponse` Streaming responses are simple, linearly ordered sequences: first a `Task` (or single `Message`), then zero or more status or artifact update events until the task reaches a terminal or interrupted state, at which point the stream closes. Implementations SHOULD avoid re-ordering events and MAY optionally resend a final `Task` snapshot before closing.

12. Custom Binding Guidelines¶

While the A2A protocol provides three standard bindings (JSON-RPC, gRPC, and HTTP+JSON/REST), implementers **MAY** create custom protocol bindings to support additional transport mechanisms or communication patterns. Custom bindings **MUST** comply with all requirements defined in Section 5 (Protocol Binding Requirements and Interoperability). This section provides additional guidelines specific to developing custom bindings.

12.1. Binding Requirements¶

Custom protocol bindings **MUST**:

1. **Implement All Core Operations**: Support all operations defined in Section 3 (A2A Protocol Operations)
2. **Preserve Data Model**: Use data structures functionally equivalent to those defined in Section 4 (Protocol Data Model)
3. **Maintain Semantics**: Ensure operations behave consistently with the abstract operation definitions
4. **Document Completely**: Provide comprehensive documentation of the binding specification

12.2. Data Type Mappings¶

Custom bindings **MUST** provide clear mappings for:

- **Protocol Buffer Types**: Define how each Protocol Buffer message type is represented
- **Timestamps**: Follow the conventions in Section 5.6.1 (Timestamps)
- **Binary Data**: Specify encoding for binary content (e.g., base64 for text-based protocols)
- **Enumerations**: Define representation of enum values (e.g., strings, integers)

12.3. Service Parameter Transmission¶

As specified in Section 3.2.6 (Service Parameters), custom protocol bindings **MUST** document how service parameters are transmitted. The binding specification **MUST** address:

1. **Transmission Mechanism**: The protocol-specific method for transmitting service parameter key-value pairs
2. **Value Constraints**: Any limitations on service parameter values (e.g., character encoding, size limits)
3. **Reserved Names**: Any service parameter names reserved by the binding itself
4. **Fallback Strategy**: What happens when the protocol lacks native header support (e.g., passing service parameters in metadata)

Example Documentation Requirements:

- **For native header support**: "Service parameters are transmitted using HTTP request headers. Service parameter keys are case-insensitive and must conform to RFC 7230. Service parameter values must be UTF-8 strings."
- **For protocols without headers**: "Service parameters are serialized as a JSON object and transmitted in the request metadata field `a2a-service-parameters`."

12.4. Error Mapping¶

Custom bindings **MUST**:

1. **Map Standard Errors**: Provide mappings for all A2A-specific error types defined in Section 3.2.2 (Error Handling)
2. **Preserve Error Information**: Ensure error details are accessible to clients
3. **Use Appropriate Codes**: Map to protocol-native error codes where applicable
4. **Document Error Format**: Specify the structure of error responses

12.5. Streaming Support¶

If the binding supports streaming operations:

1. **Define Stream Mechanism**: Document how streaming is implemented (e.g., WebSockets, long-polling, chunked encoding)
2. **Event Ordering**: Specify ordering guarantees for streaming events
3. **Reconnection**: Define behavior for connection interruption and resumption
4. **Stream Termination**: Specify how stream completion is signaled

If streaming is not supported, the binding **MUST** clearly document this limitation in the Agent Card.

12.6. Authentication and Authorization¶

Custom bindings **MUST**:

1. **Support Standard Schemes**: Implement authentication schemes declared in the Agent Card
2. **Document Integration**: Specify how credentials are transmitted in the protocol
3. **Handle Challenges**: Define how authentication challenges are communicated
4. **Maintain Security**: Follow security best practices for the transport protocol

12.7. Agent Card Declaration¶

Custom bindings **MUST** be declared in the Agent Card:

1. **Transport Identifier**: Use a URI to identify the binding (see Section 5.8)
2. **Endpoint URL**: Provide the full URL where the binding is available

Example:

```
{
  "supportedInterfaces": [
    {
      "url": "wss://agent.example.com/a2a/websocket",
      "protocolBinding": "https://example.com/bindings/websocket/v1"
    }
  ]
}
```

12.8. Interoperability Testing¶

Custom binding implementers **SHOULD**:

1. **Test Against Reference**: Verify behavior matches standard bindings
2. **Document Differences**: Clearly note any deviations from standard binding behavior
3. **Provide Examples**: Include sample requests and responses
4. **Test Edge Cases**: Verify handling of error conditions, large payloads, and long-running tasks

13. Security Considerations¶

This section consolidates security guidance and best practices for implementing and operating A2A agents. For additional enterprise security considerations, see Enterprise-Ready Features.

13.1. Data Access and Authorization Scoping¶

Implementations **MUST** ensure appropriate scope limitation based on the authenticated caller's authorization boundaries. This applies to all operations that access or list tasks and other resources.

Authorization Principles:

- Servers **MUST** implement authorization checks on every A2A Protocol Operations request
- Implementations **MUST** scope results to the caller's authorized access boundaries as defined by the agent's authorization model
- Even when `contextId` or other filter parameters are not specified in requests, implementations **MUST** scope results to the caller's authorized access boundaries
- Authorization models are agent-defined and **MAY** be based on:
  - User identity (user-based authorization)
  - Organizational roles or groups (role-based authorization)
  - Project or workspace membership (project-based authorization)
  - Organizational or tenant boundaries (multi-tenant authorization)
  - Custom authorization logic specific to the agent's domain

Operations Requiring Scope Limitation:

- `List Tasks`: **MUST** only return tasks visible to the authenticated client according to the agent's authorization model
- `Get Task`: **MUST** verify the authenticated client has access to the requested task according to the agent's authorization model
- Task-related operations (Cancel, Subscribe, Push Notification Config): **MUST** verify the client has appropriate access rights according to the agent's authorization model

Implementation Requirements:

- Authorization boundaries are defined by each agent's authorization model, not prescribed by the protocol
- Authorization checks **MUST** occur before any database queries or operations that could leak information about the existence of resources outside the caller's authorization scope
- Agents **SHOULD** document their authorization model and access control policies

See also: Section 3.1.4 List Tasks (Security Note) for operation-specific requirements.

13.2. Push Notification Security¶

When implementing push notifications, both agents (as webhook callers) and clients (as webhook receivers) have security responsibilities.

Agent (Webhook Caller) Requirements:

- Agents **MUST** include authentication credentials in webhook requests as specified in `PushNotificationConfig.authentication`
- Agents **SHOULD** implement reasonable timeout values for webhook requests (recommended: 10-30 seconds)
- Agents **SHOULD** implement retry logic with exponential backoff for failed deliveries
- Agents **MAY** stop attempting delivery after a configured number of consecutive failures
- Agents **SHOULD** validate webhook URLs to prevent SSRF (Server-Side Request Forgery) attacks:
  - Reject private IP ranges (127.0.0.0/8, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
  - Reject localhost and link-local addresses
  - Implement URL allowlists where appropriate

Client (Webhook Receiver) Requirements:

- Clients **MUST** validate webhook authenticity using the provided authentication credentials
- Clients **SHOULD** verify the task ID in the payload matches an expected task they created
- Clients **MUST** respond with HTTP 2xx status codes to acknowledge successful receipt
- Clients **SHOULD** process notifications idempotently, as duplicate deliveries may occur
- Clients **SHOULD** implement rate limiting to prevent webhook flooding
- Clients **SHOULD** use HTTPS endpoints for webhook URLs to ensure confidentiality

Configuration Security:

- Webhook URLs **SHOULD** use HTTPS to protect payload confidentiality in transit
- Authentication tokens in `PushNotificationConfig` **SHOULD** be treated as secrets and rotated periodically
- Agents **SHOULD** securely store push notification configurations and credentials
- Clients **SHOULD** use unique, single-purpose tokens for each push notification configuration

See also: Section 4.3 Push Notification Objects and Section 4.3.3 Push Notification Payload.

13.3. Extended Agent Card Access Control¶

The extended Agent Card feature allows agents to provide additional capabilities or information to authenticated clients beyond what is available in the public Agent Card.

Access Control Requirements:

- The `Get Extended Agent Card` operation **MUST** require authentication
- Agents **MUST** authenticate requests using one of the schemes declared in the public `AgentCard.securitySchemes` and `AgentCard.security` fields
- Agents **MAY** return different extended card content based on the authenticated client's identity or authorization level
- Agents **SHOULD** implement appropriate caching headers to control client-side caching of extended cards

Capability-Based Access:

- Extended cards **MAY** include additional skills not present in the public card
- Extended cards **MAY** expose more detailed capability information (e.g., rate limits, quotas)
- Extended cards **MAY** include organization-specific or user-specific configuration
- Agents **SHOULD** document which capabilities are available at different authentication levels

Security Considerations:

- Extended cards **SHOULD NOT** include sensitive information that could be exploited if leaked (e.g., internal service URLs, unmasked credentials)
- Agents **MUST** validate that clients have appropriate permissions before returning privileged information in extended cards
- Clients retrieving extended cards **SHOULD** replace their cached public Agent Card with the extended version for the duration of their authenticated session
- Agents **SHOULD** version extended cards appropriately and honor client cache invalidation

Availability Declaration:

- Agents declare extended card support via `AgentCard.capabilities.extendedAgentCard`
- When `capabilities.extendedAgentCard` is `false` or not present, the operation **MUST** return `UnsupportedOperationError`
- When support is declared but no extended card is configured, the operation **MUST** return `ExtendedAgentCardNotConfiguredError`

See also: Section 3.1.11 Get Extended Agent Card and Section 3.3.4 Capability Validation.

13.4. General Security Best Practices¶

Transport Security:

- Production deployments **MUST** use encrypted communication (HTTPS for HTTP-based bindings, TLS for gRPC)
- Implementations **SHOULD** use modern TLS configurations (TLS 1.3+ recommended) with strong cipher suites
- Agents **SHOULD** enforce HSTS (HTTP Strict Transport Security) headers when using HTTP-based bindings
- Implementations **SHOULD** disable support for deprecated SSL/TLS versions (SSLv3, TLS 1.0, TLS 1.1)

Input Validation:

- Agents **MUST** validate all input parameters before processing
- Agents **SHOULD** implement appropriate limits on message sizes, file sizes, and request complexity
- Agents **SHOULD** sanitize or validate file content types and reject unexpected media types

Credential Management:

- API keys, tokens, and other credentials **MUST** be treated as secrets
- Credentials **SHOULD** be rotated periodically
- Credentials **SHOULD** be transmitted only over encrypted connections
- Agents **SHOULD** implement credential revocation mechanisms
- Agents **SHOULD** log authentication failures and implement rate limiting to prevent brute-force attacks

Audit and Monitoring:

- Agents **SHOULD** log security-relevant events (authentication failures, authorization denials, suspicious requests)
- Agents **SHOULD** implement monitoring for unusual patterns (rapid task creation, excessive cancellations)
- Agents **SHOULD** provide audit trails for sensitive operations
- Logs **MUST NOT** include sensitive information (credentials, personal data) unless required and properly protected

Rate Limiting and Abuse Prevention:

- Agents **SHOULD** implement rate limiting on all operations
- Agents **SHOULD** return appropriate error responses when rate limits are exceeded
- Agents **MAY** implement different rate limits for different operations or user tiers

Data Privacy:

- Agents **MUST** comply with applicable data protection regulations
- Agents **SHOULD** provide mechanisms for users to request deletion of their data
- Agents **SHOULD** implement appropriate data retention policies
- Agents **SHOULD** minimize logging of sensitive or personal information

Custom Binding Security:

- Custom protocol bindings **MUST** address security considerations in their specification
- Custom bindings **SHOULD** follow the same security principles as standard bindings
- Custom bindings **MUST** document authentication integration and credential transmission

See also: Section 12.6 Authentication and Authorization (Custom Bindings).

14. IANA Considerations¶

This section provides registration templates for the A2A protocol's media type, HTTP headers, and well-known URI, intended for submission to the Internet Assigned Numbers Authority (IANA).

14.1. Media Type Registration¶

14.1.1. application/a2a+json¶

Type name: `application`

Subtype name: `a2a+json`

Required parameters: None

Optional parameters:

- None

Encoding considerations: Binary (UTF-8 encoding MUST be used for JSON text)

Security considerations: This media type shares security considerations common to all JSON-based formats as described in RFC 8259, Section 12. Additionally:

- Content MUST be validated against the A2A protocol schema before processing
- Implementations MUST sanitize user-provided content to prevent injection attacks
- File references within A2A messages MUST be validated to prevent server-side request forgery (SSRF)
- Authentication and authorization MUST be enforced as specified in Section 7 of the A2A specification
- Sensitive information in task history and artifacts MUST be protected according to applicable data protection regulations

Interoperability considerations: The A2A protocol supports multiple protocol bindings. This media type is intended for the HTTP+JSON/REST binding.

Published specification: Agent2Agent (A2A) Protocol Specification, available at: https://a2a-protocol.org/latest/specification

Applications that use this media type: AI agent platforms, agentic workflow systems, multi-agent collaboration tools, and enterprise automation systems that implement the A2A protocol for agent-to-agent communication.

Fragment identifier considerations: None

Additional information:

- **Deprecated alias names for this type:** None
- **Magic number(s):** None
- **File extension(s):** .a2a.json
- **Macintosh file type code(s):** TEXT

Person & email address to contact for further information: A2A Protocol Working Group, a2a-protocol@example.org

Intended usage: COMMON

Restrictions on usage: None

Author: A2A Protocol Working Group

Change controller: A2A Protocol Working Group

Provisional registration: No

14.2. HTTP Header Field Registrations¶

Note: The following HTTP headers represent the HTTP-based protocol binding implementation of the abstract A2A service parameters defined in Section 3.2.6. These registrations are specific to HTTP/HTTPS transports.

14.2.1. A2A-Version Header¶

Header field name: A2A-Version

Applicable protocol: HTTP

Status: Standard

Author/Change controller: A2A Protocol Working Group

Specification document: Section 3.2.5 of the A2A Protocol Specification

Related information: The A2A-Version header field indicates the A2A protocol version that the client is using. The value MUST be in the format `Major.Minor` (e.g., "0.3"). If the version is not supported by the agent, the agent returns a `VersionNotSupportedError`.

Example:

```
A2A-Version: 0.3
```

14.2.2. A2A-Extensions Header¶

Header field name: A2A-Extensions

Applicable protocol: HTTP

Status: Standard

Author/Change controller: A2A Protocol Working Group

Specification document: Section 3.2.5 of the A2A Protocol Specification

Related information: The A2A-Extensions header field contains a comma-separated list of extension URIs that the client wants to use for the request. Extensions allow agents to provide additional functionality beyond the core A2A specification while maintaining backward compatibility.

Example:

```
A2A-Extensions: https://example.com/extensions/geolocation/v1,https://standards.org/extensions/citations/v1
```

14.3. Well-Known URI Registration¶

URI suffix: agent-card.json

Change controller: A2A Protocol Working Group

Specification document: Section 8.2 of the A2A Protocol Specification

Related information: The `.well-known/agent-card.json` URI provides a standardized location for discovering an A2A agent's capabilities, supported protocols, authentication requirements, and available skills. The resource at this URI MUST return an AgentCard object as defined in Section 4.4.1 of the A2A specification.

Status: Permanent

Security considerations:

- The Agent Card MAY contain public information about an agent's capabilities and SHOULD NOT include sensitive credentials or internal implementation details
- Implementations SHOULD support HTTPS to ensure authenticity and integrity of the Agent Card
- Agent Cards MAY be signed using JSON Web Signatures (JWS) as specified in the AgentCardSignature object (Section 4.4.7)
- Clients SHOULD verify signatures when present to ensure the Agent Card has not been tampered with
- Extended Agent Cards retrieved via authenticated endpoints (Section 3.1.11) MAY contain additional information and MUST enforce appropriate access controls

Example:

```
https://agent.example.com/.well-known/agent-card.json
```

Appendix A. Migration & Legacy Compatibility¶

This appendix catalogs renamed protocol messages and objects, their legacy identifiers, and the planned deprecation/removal schedule. All legacy names and anchors MUST remain resolvable until the stated earliest removal version.

Legacy Name	Current Name	Earliest Removal Version	Notes
`MessageSendParams`	`SendMessageRequest`	>= 0.5.0	Request payload rename for clarity (request vs params)
`SendMessageSuccessResponse`	`SendMessageResponse`	>= 0.5.0	Unified success response naming
`SendStreamingMessageSuccessResponse`	`StreamResponse`	>= 0.5.0	Shorter, binding-agnostic streaming response
`SetTaskPushNotificationConfigRequest`	`CreateTaskPushNotificationConfigRequest`	>= 0.5.0	Explicit creation intent
`ListTaskPushNotificationConfigSuccessResponse`	`ListTaskPushNotificationConfigsResponse`	>= 0.5.0	Consistent response suffix removal
`GetAuthenticatedExtendedCardRequest`	`GetExtendedAgentCardRequest`	>= 0.5.0	Removed "Authenticated" from naming

Planned Lifecycle (example timeline; adjust per release strategy):

1. 0.3.x: New names introduced; legacy names documented; aliases added.
2. 0.4.x: Legacy names marked "deprecated" in SDKs and schemas; warning notes added.
3. ≥0.5.0: Legacy names eligible for removal after review; migration appendix updated.

A.1 Legacy Documentation Anchors¶

Hidden anchor spans preserve old inbound links:

Each legacy span SHOULD be placed adjacent to the current object's heading (to be inserted during detailed object section edits). If an exact numeric-prefixed anchor existed (e.g., `#414-message`), add an additional span matching that historical form if known.

A.2 Migration Guidance¶

Client Implementations SHOULD:

- Prefer new names immediately for all new integrations.
- Implement dual-handling where schemas/types permit (e.g., union type or backward-compatible decoder).
- Log a warning when receiving legacy-named objects after the first deprecation announcement release.

Server Implementations MAY:

- Accept both legacy and current request message forms during the overlap period.
- Emit only current form in responses (recommended) while providing explicit upgrade notes.

A.2.1 Breaking Change: Kind Discriminator Removed¶

Version 1.0 introduces a breaking change in how polymorphic objects are represented in the protocol. This affects `Part` types and streaming event types.

Legacy Pattern (v0.3.x): Objects used an inline `kind` field as a discriminator to identify the object type:

Example 1 - TextPart:

```
{
  "kind": "text",
  "text": "Hello, world!"
}
```

Example 2 - FilePart:

```
{
  "kind": "file",
  "file": {
    "name": "diagram.png",
    "mimeType": "image/png",
    "fileWithBytes": "iVBORw0KGgo..."
  }
}
```

Current Pattern (v1.0): Objects now use the **JSON member name** itself to identify the type. The member name acts as the discriminator, and the value structure depends on the specific type:

Example 1 - TextPart:

```
{
  "text": "Hello, world!"
}
```

Example 2 - FilePart:

```
{
  "raw": "iVBORw0KGgo...",
  "filename": "diagram.png",
  "mediaType": "image/png"
}
```

Affected Types:

1. **Part Union Types**:
2. **TextPart**:
   - **Legacy:** `{ "kind": "text", "text": "..." }`
   - **Current:** `{ "text": "..." }` (member presence acts as discriminator)
3. **FilePart**:
   - **Legacy:** `{ "kind": "file", "file": { "name": "...", "mimeType": "...", "fileWithBytes": "..." } }`
   - **Current:** `{ "raw": "...", "filename": "...", "mediaType": "..." }` (or `url` instead of `raw`)
4. **DataPart**:
   - **Legacy:** `{ "kind": "data", "data": {...} }`
   - **Current:** `{ "data": {...}, "mediaType": "application/json" }`
5. **Streaming Event Types**:
6. **TaskStatusUpdateEvent**:
   - **Legacy:** `{ "kind": "status-update", "taskId": "...", "status": {...} }`
   - **Current:** `{ "statusUpdate": { "taskId": "...", "status": {...} } }`
7. **TaskArtifactUpdateEvent**:
   - **Legacy:** `{ "kind": "artifact-update", "taskId": "...", "artifact": {...} }`
   - **Current:** `{ "artifactUpdate": { "taskId": "...", "artifact": {...} } }`

Migration Strategy:

For **Clients** upgrading from pre-0.3.x:

1. Update parsers to expect wrapper objects with member names as discriminators
2. When constructing requests, use the new wrapper format
3. Implement version detection based on the agent's `protocolVersions` in the `AgentCard`
4. Consider maintaining backward compatibility by detecting and handling both formats during a transition period

For **Servers** upgrading from pre-0.3.x:

1. Update serialization logic to emit wrapper objects
2. **Breaking:** The `kind` field is no longer part of the protocol and should not be emitted
3. Update deserialization to expect wrapper objects with member names
4. Ensure the `AgentCard` declares the correct `protocolVersions` (e.g., ["1.0"] or later)

Rationale:

This change aligns with modern API design practices and Protocol Buffers' `oneof` semantics, where the field name itself serves as the type discriminator. This approach:

- Reduces redundancy (no need for both a field name and a `kind` value)
- Aligns JSON-RPC and gRPC representations more closely
- Simplifies code generation from schema definitions
- Eliminates the need for representing inheritance structures in schema languages
- Improves type safety in strongly-typed languages

A.2.2 Breaking Change: Extended Agent Card Field Relocated¶

Version 1.0 relocates the extended agent card capability from a top-level field to the capabilities object for architectural consistency.

Legacy Structure (pre-1.0):

```
{
  "supportsExtendedAgentCard": true,
  "capabilities": {
    "streaming": true
  }
}
```

Current Structure (1.0+):

```
{
  "capabilities": {
    "streaming": true,
    "extendedAgentCard": true
  }
}
```

Proto Changes:

- Removed: `AgentCard.supports_extended_agent_card` (field 13)
- Added: `AgentCapabilities.extended_agent_card` (field 5)

Migration Steps:

For **Agent Implementations**:

1. Remove `supportsExtendedAgentCard` from top-level AgentCard
2. Add `extendedAgentCard` to `capabilities` object
3. Update validation: `agentCard.capabilities?.extendedAgentCard`

For **Client Implementations**:

1. Update capability checks: `agentCard.capabilities?.extendedAgentCard`
2. Temporary fallback (transition period):

```
const supported = agentCard.capabilities?.extendedAgentCard ||
                  agentCard.supportsExtendedAgentCard;
```

1. Remove fallback after agent ecosystem migrates

For **SDK Developers**:

1. Regenerate code from updated proto
2. Update type definitions
3. Document breaking change in release notes

Rationale:

All optional features enabling specific operations (`streaming`, `pushNotifications`) reside in `AgentCapabilities`. Moving `extendedAgentCard` achieves:

- Architectural consistency
- Improved discoverability
- Semantic correctness (it is a capability)

A.3 Future Automation¶

Once the proto→schema generation pipeline lands, this appendix will be partially auto-generated (legacy mapping table sourced from a maintained manifest). Until then, edits MUST be manual and reviewed in PRs affecting `a2a.proto`.

Appendix B. Relationship to MCP (Model Context Protocol)¶

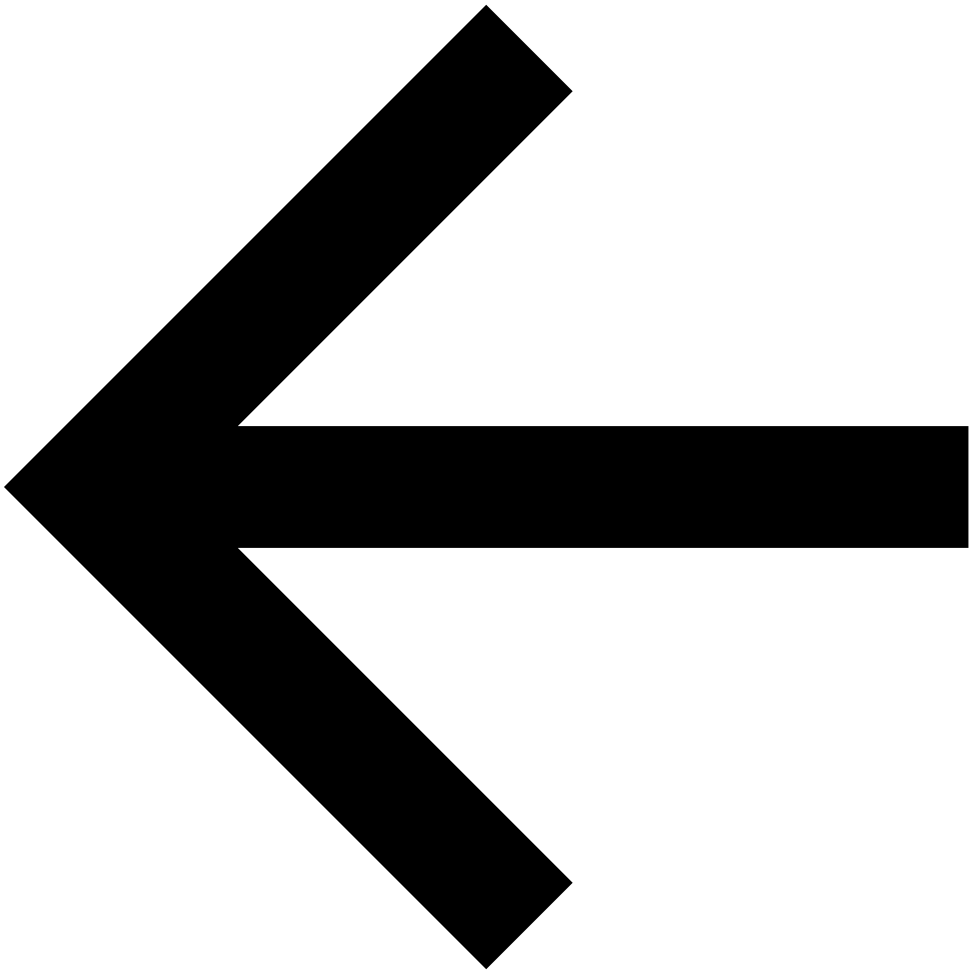
A2A and MCP are complementary protocols designed for different aspects of agentic systems:

- **Model Context Protocol (MCP):** Focuses on standardizing how AI models and agents connect to and interact with **tools, APIs, data sources, and other external resources.** It defines structured ways to describe tool capabilities (like function calling in LLMs), pass inputs, and receive structured outputs. Think of MCP as the

"how-to" for an agent to *use* a specific capability or access a resource.
- **Agent2Agent Protocol (A2A):** Focuses on standardizing how independent, often opaque, **AI agents communicate and collaborate with each other as peers.** A2A provides an application-level protocol for agents to discover each other, negotiate interaction modalities, manage shared tasks, and exchange conversational context or complex results. It's about how agents *partner* or *delegate* work.

How they work together: An A2A Client agent might request an A2A Server agent to perform a complex task. The Server agent, in turn, might use MCP to interact with several underlying tools, APIs, or data sources to gather information or perform actions necessary to fulfill the A2A task.

For a more detailed comparison, see the A2A and MCP guide.



Previous
Extension & Binding Governance
Next
What's New in v1.0